Unity Pro
Motion Function Blocks
Block Library

07/2016

35010605.10

www.schneider-electric.com

Schneider Electric

The information provided in this documentation contains general descriptions and/or technical characteristics of the performance of the products contained herein. This documentation is not intended as a substitute for and is not to be used for determining suitability or reliability of these products for specific user applications. It is the duty of any such user or integrator to perform the appropriate and complete risk analysis, evaluation and testing of the products with respect to the relevant specific application or use thereof. Neither Schneider Electric nor any of its affiliates or subsidiaries shall be responsible or liable for misuse of the information contained herein. If you have any suggestions for improvements or amendments or have found errors in this publication, please notify us.

No part of this document may be reproduced in any form or by any means, electronic or mechanical, including photocopying, without express written permission of Schneider Electric.

All pertinent state, regional, and local safety regulations must be observed when installing and using this product. For reasons of safety and to help ensure compliance with documented system data, only the manufacturer should perform repairs to components.

When devices are used for applications with technical safety requirements, the relevant instructions must be followed.

Failure to use Schneider Electric software or approved software with our hardware products may result in injury, harm, or improper operating results.

Failure to observe this information can result in injury or equipment damage.

© 2016 Schneider Electric. All Rights Reserved.

Table of Contents

Safety Information 5
About the Book 9
Part I General 11
Chapter 1 Block Types and their Applications 13
Block Types 14
FFB Structure 16
EN and ENO 19
Chapter 2 Block Availability on the Various Hardware Platforms . . 23
Availability of Blocks on the Various Servodrives 23
Part II MFB blocks 25
Chapter 3 The RefAxis parameter 27
The Axis_Ref 28
The status chart of the axis 29
Chapter 4 Motion Function Block 31
MFB Blocks and Basic Parameters 33
CAN_HANDLER 38
MC_READPARAMETER 40
MC_WRITEPARAMETER 43
MC_READACTUALPOSITION 45
MC_READACTUALVELOCITY 47
MC_READACTUALTORQUE 49
MC_TORQUECONTROL 51
MC_RESET 53
MC_STOP 55
MC_POWER 58
MC_MOVEABSOLUTE 60
MC_MOVERELATIVE 62
MC_MOVEADDITIVE 64
MC_MOVEVELOCITY 68
MC_JOG 70
MC_READAXISERROR 74
MC_READSTATUS 78
MC_HOME 82
LXM_GEARPOS 84

LXM_GearPosS 87
TE_UPLOADDRIVEPARAM 90
TE_DOWNLOADDRIVEPARAM 93
LXM_UPLOADMTASK 96
LXM_DOWNLOADMTASK 99
LXM_STARTMTASK 101
Units and Servodrives 103
Messaging Service 105
Memory Size 107
Appendices 111
Appendix A Error Codes and Values 113
Tables of Error Codes for the Motion Function Block Library 113
Appendix B MFB performances 117
MFB Performance Table 117
Glossary 121
Index 123

Safety Information

Important Information

NOTICE

Read these instructions carefully, and look at the equipment to become familiar with the device before trying to install, operate, service, or maintain it. The following special messages may appear throughout this documentation or on the equipment to warn of potential hazards or to call attention to information that clarifies or simplifies a procedure.



The addition of this symbol to a “Danger” or “Warning” safety label indicates that an electrical hazard exists which will result in personal injury if the instructions are not followed.



This is the safety alert symbol. It is used to alert you to potential personal injury hazards. Obey all safety messages that follow this symbol to avoid possible injury or death.

⚠ DANGER

DANGER indicates a hazardous situation which, if not avoided, **will result in** death or serious injury.

⚠ WARNING

WARNING indicates a hazardous situation which, if not avoided, **could result in** death or serious injury.

⚠ CAUTION

CAUTION indicates a hazardous situation which, if not avoided, **could result** in minor or moderate injury.

NOTICE

NOTICE is used to address practices not related to physical injury.

PLEASE NOTE

Electrical equipment should be installed, operated, serviced, and maintained only by qualified personnel. No responsibility is assumed by Schneider Electric for any consequences arising out of the use of this material.

A qualified person is one who has skills and knowledge related to the construction and operation of electrical equipment and its installation, and has received safety training to recognize and avoid the hazards involved.

BEFORE YOU BEGIN

Do not use this product on machinery lacking effective point-of-operation guarding. Lack of effective point-of-operation guarding on a machine can result in serious injury to the operator of that machine.

⚠ WARNING

UNGUARDED EQUIPMENT

- Do not use this software and related automation equipment on equipment which does not have point-of-operation protection.
- Do not reach into machinery during operation.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

This automation equipment and related software is used to control a variety of industrial processes. The type or model of automation equipment suitable for each application will vary depending on factors such as the control function required, degree of protection required, production methods, unusual conditions, government regulations, etc. In some applications, more than one processor may be required, as when backup redundancy is needed.

Only you, the user, machine builder or system integrator can be aware of all the conditions and factors present during setup, operation, and maintenance of the machine and, therefore, can determine the automation equipment and the related safeties and interlocks which can be properly used. When selecting automation and control equipment and related software for a particular application, you should refer to the applicable local and national standards and regulations. The National Safety Council's Accident Prevention Manual (nationally recognized in the United States of America) also provides much useful information.

In some applications, such as packaging machinery, additional operator protection such as point-of-operation guarding must be provided. This is necessary if the operator's hands and other parts of the body are free to enter the pinch points or other hazardous areas and serious injury can occur. Software products alone cannot protect an operator from injury. For this reason the software cannot be substituted for or take the place of point-of-operation protection.

Ensure that appropriate safeties and mechanical/electrical interlocks related to point-of-operation protection have been installed and are operational before placing the equipment into service. All interlocks and safeties related to point-of-operation protection must be coordinated with the related automation equipment and software programming.

NOTE: Coordination of safeties and mechanical/electrical interlocks for point-of-operation protection is outside the scope of the Function Block Library, System User Guide, or other implementation referenced in this documentation.

START-UP AND TEST

Before using electrical control and automation equipment for regular operation after installation, the system should be given a start-up test by qualified personnel to verify correct operation of the equipment. It is important that arrangements for such a check be made and that enough time is allowed to perform complete and satisfactory testing.

⚠ WARNING

EQUIPMENT OPERATION HAZARD

- Verify that all installation and set up procedures have been completed.
- Before operational tests are performed, remove all blocks or other temporary holding means used for shipment from all component devices.
- Remove tools, meters, and debris from equipment.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

Follow all start-up tests recommended in the equipment documentation. Store all equipment documentation for future references.

Software testing must be done in both simulated and real environments.

Verify that the completed system is free from all short circuits and temporary grounds that are not installed according to local regulations (according to the National Electrical Code in the U.S.A, for instance). If high-potential voltage testing is necessary, follow recommendations in equipment documentation to prevent accidental equipment damage.

Before energizing equipment:

- Remove tools, meters, and debris from equipment.
- Close the equipment enclosure door.
- Remove all temporary grounds from incoming power lines.
- Perform all start-up tests recommended by the manufacturer.

OPERATION AND ADJUSTMENTS

The following precautions are from the NEMA Standards Publication ICS 7.1-1995 (English version prevails):

- Regardless of the care exercised in the design and manufacture of equipment or in the selection and ratings of components, there are hazards that can be encountered if such equipment is improperly operated.
- It is sometimes possible to misadjust the equipment and thus produce unsatisfactory or unsafe operation. Always use the manufacturer's instructions as a guide for functional adjustments. Personnel who have access to these adjustments should be familiar with the equipment manufacturer's instructions and the machinery used with the electrical equipment.
- Only those operational adjustments actually required by the operator should be accessible to the operator. Access to other controls should be restricted to prevent unauthorized changes in operating characteristics.

About the Book

At a Glance

Document Scope

This manual presents the Motion Function Block (MFB) library using Unity Pro.

Validity Note

This documentation is valid for Unity Pro 11.1 or later.

Related Documents

Title of Documentation	Reference Number
Unity Pro, Operating Modes	33003101 (English), 33003102 (French), 33003103 (German), 33003696 (Italian), 33003104 (Spanish), 33003697 (Chinese)
Unity Pro Communication, Block Library	33002527 (English), 33002528 (French), 33002529 (German), 33003682 (Italian), 33002530 (Spanish), 33003683 (Chinese)
Modicon M580, BMECXM CANopen module, User Manual	EIO0000002129 (English), EIO0000002130 (French), EIO0000002131 (German), EIO0000002132 (Italian), EIO0000002133 (Spanish), EIO0000002134 (Chinese)
Modicon M340 with Unity Pro, CANopen, User Manual	35013944 (English), 35013945 (French), 35013946 (German), 35013948 (Italian), 35013947 (Spanish), 35013949 (Chinese)

You can download these technical publications and other technical information from our website at http://download.schneider-electric.com

Product Related Information

⚠ WARNING

UNINTENDED EQUIPMENT OPERATION

The application of this product requires expertise in the design and programming of control systems. Only persons with such expertise should be allowed to program, install, alter, and apply this product.

Follow all local and national safety codes and standards.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

Part I
General

Overview

This section contains general information about the Motion Function Blocks (MFB).

What Is in This Part?

This part contains the following chapters:

Chapter	Chapter Name	Page
1	Block Types and their Applications	13
2	Block Availability on the Various Hardware Platforms	23

Chapter 1
Block Types and their Applications

Overview

This chapter describes the different block types and their applications.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Block Types	14
FFB Structure	16
EN and ENO	19

Block Types

Block Types

Different block types are used in Unity Pro. The general term for the block types is FFB.

There are the following types of block:

- Elementary Function (EF)
- Elementary Function Block (EFB)
- Derived Function Block (DFB)
- Procedure

NOTE: Motion Function Blocks are not available on the Quantum platform.

Elementary Function

Elementary functions (EF) have no internal status and one output only. If the input values are the same, the output value is the same for the executions of the function, for example the addition of two values gives the same result at every execution.

An elementary function is represented in the graphical languages (FBD and LD) as a block frame with inputs and an output. The inputs are represented on the left and the outputs on the right of the block frame. The name of the function, that is the function type, is shown in the center of the block frame.

The number of inputs can be increased with some elementary functions.

⚠ CAUTION

UNEXPECTED BEHAVIOR OF EQUIPMENT

For Unity Pro V4.0 and earlier versions, do not use links to connect function blocks outputs, when your application relies on persistent output data of an EF.

Failure to follow these instructions can result in injury or equipment damage.

NOTE: With Unity Pro V4.0 and earlier versions the deactivation of an EF (EN=0) causes links connected to its Input/Output to be reset. To transfer the state of the signal do not use a link. A variable must be connected to the EF's output and must be used to connect the input of the element. With Unity Pro V4.1 and later versions you can maintain the output links even if an EF is deactivated by activating the option **Maintain output links on disabled EF (EN=0)** via the menu **Tools → Program → Languages → Common**.

Elementary Function Block

Elementary function blocks (EFB) have an internal status. If the inputs have the same values, the value on the outputs can have another value during the individual executions. For example, with a counter, the value on the output is incremented.

An elementary function block is represented in the graphical languages (FBD and LD) as a block frame with inputs and outputs. The inputs are represented on the left and the outputs on the right of the block frame. The name of the function block, that is the function block type, is shown in the center of the block frame. The instance name is displayed above the block frame.

Derived Function Block

Derived function blocks (DFBs) have the same properties as elementary function blocks. They are created by the user in the programming languages FBD, LD, IL and/or ST.

Procedure

Procedures are functions with several outputs. They have no internal state.

The only difference from elementary functions is that procedures can have more than one output and they support variables of the `VAR_IN_OUT` data type.

Procedures do not return a value.

Procedures are a supplement to IEC 61131-3 and must be enabled explicitly.

There is no visual difference between procedures and elementary functions.

⚠ CAUTION

UNEXPECTED BEHAVIOR OF EQUIPMENT

For Unity Pro V4.0 and earlier versions, do not use links to connect function blocks outputs, when your application relies on persistent output data of an EF.

Failure to follow these instructions can result in injury or equipment damage.

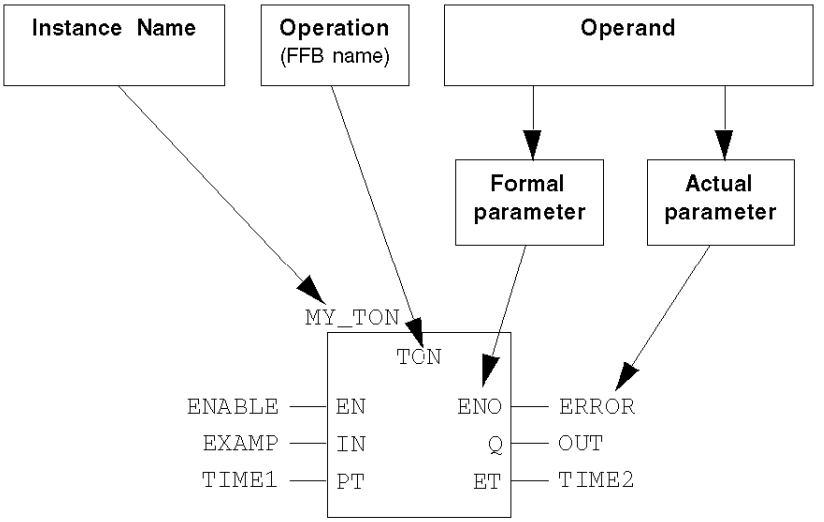
NOTE: With Unity Pro V4.0 and earlier versions the deactivation of an EF (EN=0) causes links connected to its Input/Output to be reset. To transfer the state of the signal do not use a link. A variable must be connected to the EF's output and must be used to connect the input of the element. With Unity Pro V4.1 and later versions you can maintain the output links even if an EF is deactivated by activating the option **Maintain output links on disabled EF (EN=0)** via the menu **Tools → Program → Languages → Common**.

FFB Structure

Structure

Each FFB is made up of an operation (name of the FFB), the operands are required for the operation (formal and actual parameters) and an instance name for elementary/derived function blocks.

Call of a function block in the FBD programming language:



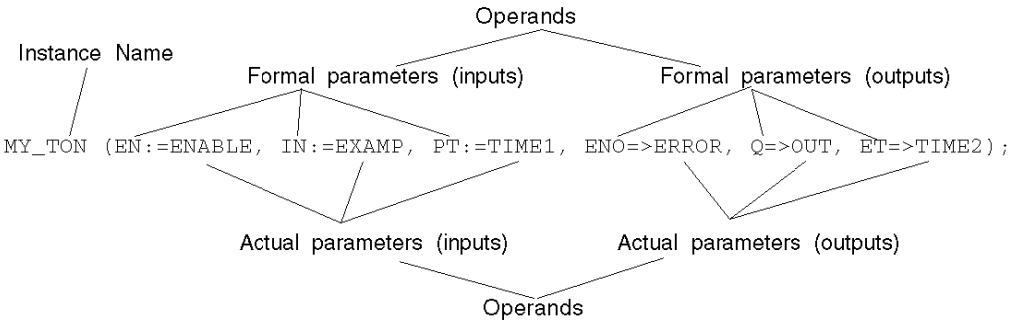
⚠ CAUTION

UNEXPECTED APPLICATION BEHAVIOR

Do not call several times the same block instance within a PLC cycle

Failure to follow these instructions can result in injury or equipment damage.

Formal call of a function block in the ST programming language:



Operation

The operation determines which function is to be executed with the FFB, e.g. shift register, conversion operations.

Operand

The operand specifies what the operation is to be executed with. With FFBs, this consists of formal and actual parameters.

Formal/actual parameters

Inputs and outputs are required to transfer values to or from an FFB. These are called formal parameters.

Objects are linked to formal parameters; these objects contain the current process states. They are called actual parameters.

At program runtime, the values from the process are transferred to the FFB via the actual parameters and then output again after processing.

The data type of the actual parameters must match the data type of the input/output (formal parameters). The only exceptions are generic inputs/outputs whose data type is determined by the actual parameter. If the actual parameters consist of literals, a suitable data type is selected for the function block.

FFB Call in IL/ST

In text languages IL and ST, FFBs can be called in formal and in informal form. Details can be found in the *Reference manual*.

Example of a formal function call:

```
out:=LIMIT (MN:=0, IN:=var1, MX:=5);
```

Example of an informal function call:

```
out:=LIMIT (0, var1, 5);
```

NOTE: The use of `EN` and `ENO` is only possible for formal calls.

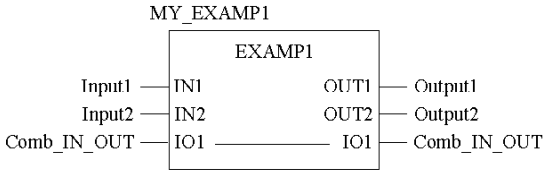
`VAR_IN_OUT` variable

FFBs are often used to read a variable at an input (input variables), to process it and to output the altered values of the **same** variable (output variables).

This special type of input/output variable is also called a `VAR_IN_OUT` variable.

The input and output variable are linked in the graphic languages (FBD and LD) using a line showing that they belong together.

Function block with `VAR_IN_OUT` variable in FBD:



Function block with `VAR_IN_OUT` variable in ST:

```
MY_EXAMP1 (IN1:=Input1, IN2:=Input2, IO1:=Comb_IN_OUT,
           OUT1=>Output1, OUT2=>Output2);
```

The following points must be considered when using FFBs with `VAR_IN_OUT` variables:

- The `VAR_IN_OUT` inputs must be assigned a variable.
- Literals or constants cannot be assigned to `VAR_IN_OUT` inputs/outputs.

The following additional limitations apply to the graphic languages (FBD and LD):

- When using graphic connections, `VAR_IN_OUT` outputs can only be connected with `VAR_IN_OUT` inputs.
- Only one graphical link can be connected to a `VAR_IN_OUT` input/output.
- Different variables/variable components can be connected to the `VAR_IN_OUT` input and the `VAR_IN_OUT` output. In this case the value of the variables/variable component on the input is copied to the output variables/variable component.
- No negations can be used on `VAR_IN_OUT` inputs/outputs.
- A combination of variable/address and graphic connections is not possible for `VAR_IN_OUT` outputs.

EN and ENO

Description

An EN input and an ENO output can be configured for the FFBs.

If the value of EN is equal to "0" when the FFB is invoked, the algorithms defined by the FFB are not executed and ENO is set to "0".

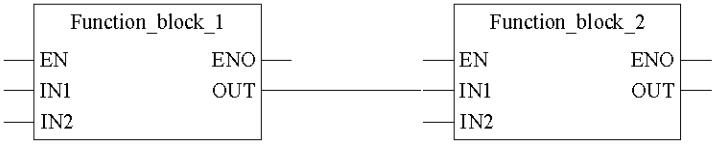
If the value of EN is equal to "1" when the FFB is invoked, the algorithms defined by the FFB will be executed. After the algorithms have been executed successfully, the value of ENO is set to "1". If certain error conditions are detected when executing these algorithms, ENO is set to "0".

If the EN pin is not assigned a value, when the FFB is invoked, the algorithm defined by the FFB is executed (same as if EN equals to "1"), Please refer to *Maintain output links on disabled EF (see Unity Pro, Operating Modes)*.

If the algorithms are executed successfully, then value of ENO is set to "1", else ENO is set to "0".

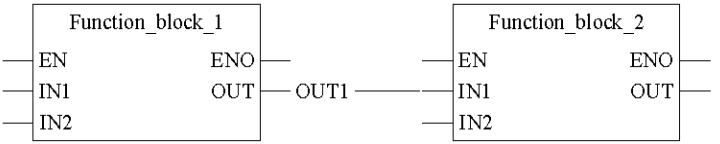
If ENO is set to "0" (caused by EN=0 or a detected error condition during execution or unsuccessful algorithm execution):

- Function blocks
  - EN/ENO handling with function blocks that (only) have one link as an output parameter:



If EN from FunctionBlock_1 is set to "0", the output connection OUT from Function-Block_1 retains the status it had in the last correctly executed cycle.

  - EN/ENO handling with function blocks that have one variable and one link as output parameters:



If EN from FunctionBlock_1 is set to "0", the output connection OUT from Function-Block_1 retains the status it had in the last correctly executed cycle. The variable OUT1 on the same pin, either retains its previous status or can be changed externally without influencing the connection. The variable and the link are saved independently of each other.

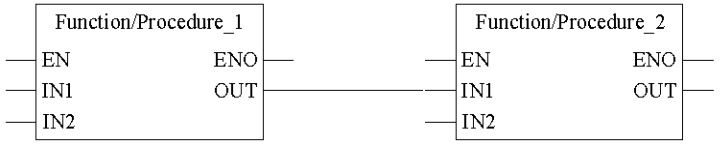
- Functions/Procedures

⚠ CAUTION
UNEXPECTED BEHAVIOR OF EQUIPMENT For Unity Pro V4.0 and earlier versions, do not use links to connect function blocks outputs, when your application relies on persistent output data of an EF. **Failure to follow these instructions can result in injury or equipment damage.**

NOTE: With Unity Pro V4.0 and earlier versions the deactivation of an EF (EN=0) causes links connected to its Input/Output to be reset. To transfer the state of the signal do not use a link. A variable must be connected to the EF's output and must be used to connect the input of the element. With Unity Pro V4.1 and later versions you can maintain the output links even if an EF is deactivated by activating the option **Maintain output links on disabled EF (EN=0)** via the menu **Tools → Program → Languages → Common**.

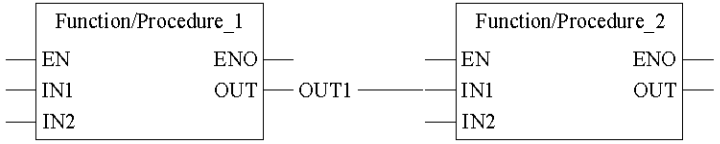
As defined in IEC61131-3, the outputs from deactivated functions (`EN`-input set to "0") is undefined. (The same applies to procedures.)
Here is an explanation of the output status in this case:

- EN/ENO handling with functions/procedures that (only) have one link as an output parameter:



  If `EN` from `Function/Procedure_1` is set to "0", the output connection `OUT` from `Function/Procedure_1` is also set to "0".

- EN/ENO handling with function blocks that have one variable and one link as output parameters:



  If `EN` from `Function/Procedure_1` is set to "0", the output connection `OUT` from `Function/Procedure_1` is also set to "0". The variable `OUT1` on the same pin, either retains its previous status or can be changed externally without influencing the connection. The variable and the link are saved independently of each other.

The output behavior of the FFBs does not depend on whether the FFBs are called up without `EN`/`ENO` or with `EN`=1.

Conditional/Unconditional FFB Call

"Unconditional" or "conditional" calls are possible with each FFB. The condition is realized by pre-linking the input EN.

- EN connected
  conditional calls (the FFB is only processed if EN = 1)
- EN shown, hidden, and marked TRUE, or shown and not occupied
  unconditional calls (FFB is processed independent from EN)

NOTE: For disabled function blocks (EN = 0) with an internal time function (e.g. DELAY), time seems to keep running, since it is calculated with the help of a system clock and is therefore independent of the program cycle and the release of the block.

⚠ CAUTION
UNEXPECTED APPLICATION EQUIPMENT
Do not disable function blocks with internal time function during their operation.
Failure to follow these instructions can result in injury or equipment damage.

Note for IL and ST

The use of EN and ENO is only possible in the text languages for a formal FFB call, e.g.

```
MY_BLOCK (EN:=enable, IN1:=var1, IN2:=var2,
ENO=>error, OUT1=>result1, OUT2=>result2);
```

Assigning the variables to ENO must be done with the operator =>.

With an informal call, EN and ENO cannot be used.

Chapter 2
Block Availability on the Various Hardware Platforms

Availability of Blocks on the Various Servodrives

Motion Function Blocks

The motion function blocks availability can be found in the following tables.

Type	Block name	ATV 31 ATV312 (7.)	ATV 32	ATV 71	Lexium 32, 32i	Lexium 05	Lexium 15 HP, MP, LP	IclA IFA, IFE, IFS
PLCopen motioncontrol V1.1	MC_ReadParameter	X	X	X	X	X	X	X
	MC_WriteParameter	X	X	X	X	X	X	X
	MC_ReadActualPosition				X	X	X	X
	MC_ReadActualVelocity (1.)	X	X	X	X	X	X	X
	MC_Reset	X	X	X	X	X	X	X
	MC_Stop	X	X	X	X	X	X	X
	MC_Power	X	X	X	X	X	X	X
	MC_MoveAbsolute				X	X	X	X
	MC_MoveRelative				X	X	X	
	MC_MoveAdditive				X	X		X
	MC_Home				X	X	X	X
	MC_MoveVelocity	X	X	X	X	X	X	X
	MC_ReadAxisError	X	X	X	X	X	X	X
	MC_ReadStatus	X	X	X	X	X	X	X
	MC_TorqueControl (1.)			X	X	X	X (3.)	
	MC_ReadActualTorque (1.)	X	X	X	X	X	X	
	MC_Jog (2.)				X	X	X (3.), except 15 LP	X

Type	Block name	ATV 31 ATV312 (7.)	ATV 32	ATV 71	Lexium 32, 32i	Lexium 05	Lexium 15 HP, MP, LP	IclA IFA, IFE, IFS
Parameter set save and restore functions for management of recipes or replacement of faulty servodrives	TE_UploadDriveParam	X	X	X	X(6.), except 32i	X	X	X
	TE_DownloadDriveParam	X	X	X	X(6.), except 32i	X	X	X
Advanced functions for the **Lexium**	Lxm_GearPos					X(4.)	X(5.)	
	Lxm_GearPosS				X	X(4.)	X(5.)	
	Lxm_UploadMTask						X	
	Lxm_DownloadMTask						X	
	Lxm_StartMTask				X		X	
System function	CAN_Handler	X	X	X	X	X	X	X

1. PLCopen V0.99 extension part 2
2. Not PLCopen standard
3. Only for firmware version >= 6.73
4. Only for firmware version >= 1.403
5. Only for firmware version >= 2.36
6. The parameter list is a Lexium32Advanced drive parameter list
7. Through an ATV 31 V1.7 CANopen Device configuration.

Part II
MFB blocks

Subject of this Section

This section describes the MotionFunctionBlock library's functions and elementary function blocks.

What Is in This Part?

This part contains the following chapters:

Chapter	Chapter Name	Page
3	The RefAxis parameter	27
4	Motion Function Block	31

Chapter 3
The RefAxis parameter

Subject of this Chapter

This chapter describes in detail the RefAxis parameter.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
The Axis_Ref	28
The status chart of the axis	29

The Axis_Ref

Description

An axis is defined using an AXIS_REF type object.

From the user's viewpoint, an AXIS_REF object is:

- A structured variable with all the data needed by the MFBs to work with an axis (logical status, mapping, messaging status, etc.),
- A variable created automatically during the creation of the axis,
- A variable to be addressed to MFB blocks.

The AXIS_REF object is represented by a DDT type structure that contains public data, which can therefore be modified by the axis configuration, as well as data that is not visible and that cannot be modified.

The visible data in AXIS_REF is:

Data	Type	Description
AxisReady	BOOL	Provides information on the initialization of Axis_Ref, as well as the availability of the device on the network.
AxisType	UINT	Servodrive type 1=Lexium, 2=Ifx, 3=ATV31, etc.
AxisReference	UINT	Device reference 1=MHDA1004, 2=MHDA1008, etc. Used to check data when using UploadParam and DownloadParam functions.
PLCTask	UINT	Task identification (1=MAST, 2=FAST).
NetworkType	UINT	Reserved
AxisMajorVersion	UINT	Integer part of the minimum version that the servodrive must have (e.g. the digit 6 in the number 6.43).
AxisMinorVersion	UINT	Decimal part of the minimum version that the configured servodrive must have (e.g. the number 43 in 6.43).
AxisMajorVersionRead	UINT	Integer part of the minimum version of the present servodrive (e.g. the number 6 in the number 6.5).
AxisMinorVersionRead	UINT	Decimal part of the minimum version that the servodrive has (e.g. the number 5 in the number 6.5).

The status chart of the axis

At a Glance

The inverters and servodrives are managed by the standard DRIVECOM state diagram. The Motion Function Blocks defines the behavior of the axis at a high level by a standard PLCopen state diagram. The MFB commands act on the axis' state diagram. The axis is always in one of the defined states. Any motion command is a transition that changes the states of the axis and, as a consequence, the way the current motion is computed.

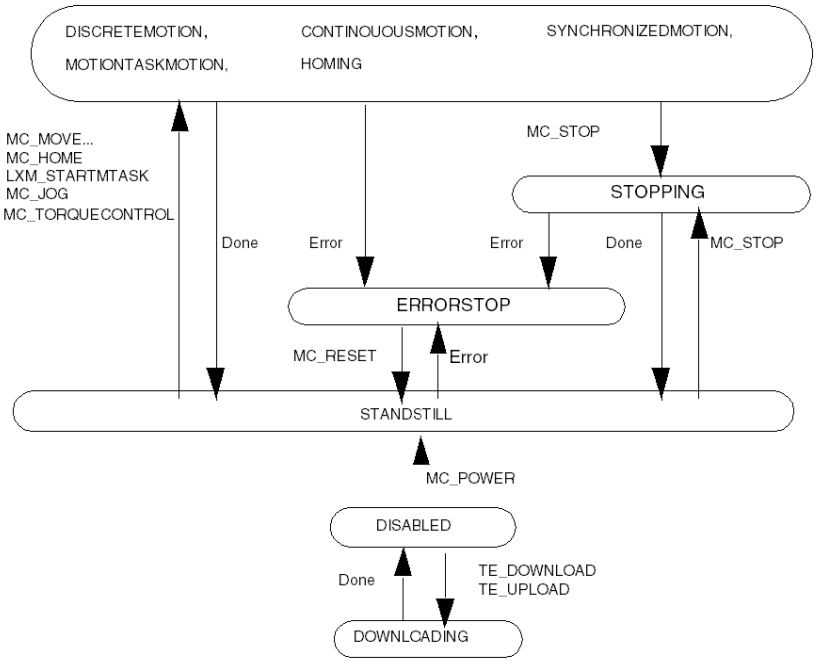
There are four main status values in this chart:

- disabled
- standstill
- errorstop
- stopping.

Other status values are described below.

The Chart

The Axis_Ref state chart is shown below:



Status Values

The following table describes the status values:

Status	Corresponds to
Disabled	Idle or initial status of the axis.
Standstill	Waiting status of the axis, it is powered up and is error-free.
Discrete Motion	Discrete motion in progress.
Continuous Motion	Continuous motion in progress.
Synchronized Motion	Synchronized motion in progress.
MotionTask Motion	Current motion task program.
Downloading	Parameters or motion tasks currently being transferred between the PLC and the servodrive memory.
Homing	The axis is executing a homing.
Stopping	Valid during the execution of the axis' MC_STOP, the Done and the non-execution of MC_STOP brings the Axis_Ref to the Standstill status.
Errorstop	Valid during the axis stop due to an error, before the error has been acknowledged.

Chapter 4
Motion Function Block

Overview

This chapters presents the various blocks in the Motion Function Blocks library.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
MFB Blocks and Basic Parameters	33
CAN_HANDLER	38
MC_READPARAMETER	40
MC_WRITEPARAMETER	43
MC_READACTUALPOSITION	45
MC_READACTUALVELOCITY	47
MC_READACTUALTORQUE	49
MC_TORQUECONTROL	51
MC_RESET	53
MC_STOP	55
MC_POWER	58
MC_MOVEABSOLUTE	60
MC_MOVERELATIVE	62
MC_MOVEADDITIVE	64
MC_MOVEVELOCITY	68
MC_JOG	70
MC_READAXISERROR	74
MC_READSTATUS	78
MC_HOME	82
LXM_GEARPOS	84
LXM_GearPosS	87
TE_UPLOADDRIVEPARAM	90
TE_DOWNLOADDRIVEPARAM	93
LXM_UPLOADMTASK	96
LXM_DOWNLOADMTASK	99

Topic	Page
LXM_STARTMTASK	101
Units and Servodrives	103
Messaging Service	105
Memory Size	107

MFB Blocks and Basic Parameters

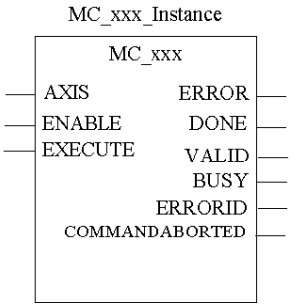
Overview

Most blocks use the same input and output parameters, deemed basic.

The operating principle of the basic input and output parameters is explained below.

Representation in FBD

Representation:



Description of the Standard Input Parameters

The following table describes the input parameters:

Parameter	Type	Comment
AXIS	AXIS_REF	Axis_Ref *(see page 28)* type object, which defines the device
ENABLE	BOOL	When ENABLE is TRUE, the parameters are taken into account and the function is executed. As soon as ENABLE is FALSE, the output parameters, ERROR, DONE, and COMMANDABORTED, are immediately set to FALSE.
EXECUTE	BOOL	On an EXECUTE rising edge, the parameters are taken into account and the function is executed. When EXECUTE is TRUE, the output parameters, ERROR, DONE, COMMANDABORTED, and BUSY are controlled by the block. When EXECUTE is FALSE, the BUSY output parameter is TRUE until the end of the block execution. As soon as one of the ERROR, DONE, or COMMANDABORTED output parameter changes to TRUE, they all become FALSE.

Description of the Standard Output Parameters

The following table describes the output parameters:

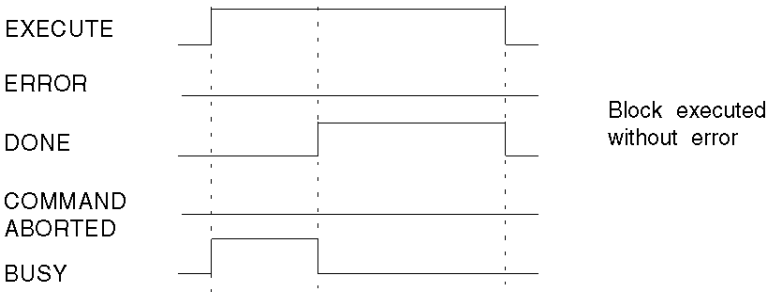
Parameter	Type	Comment
ERROR	BOOL	ERROR is TRUE when an execution error is detected by the function block.
DONE	BOOL	DONE is TRUE when the execution of the function is completed.
VALID	BOOL	VALID is TRUE when the others are enabled.
BUSY	BOOL	BUSY is TRUE to indicate that the execution of the function block is in progress. BUSY is set to TRUE when EXECUTE switches to TRUE (on a rising edge), and is set to FALSE when one of the following status values, DONE, ERROR, or COMMANDABORTED switches to TRUE.
ERRORID	INT	Error identifier.
COMMANDABORTED	BOOL	COMMANDABORTED is TRUE when the execution of the block is cancelled. This cancellation is due to the execution of another command.

NOTE: The type of the output parameter ErrorId (Error Identifier *(see page 113)*) is UDINT for the blocks that use messaging (MC_READ..., MC_WRITEPARAMETER and CAN_HANDLER).

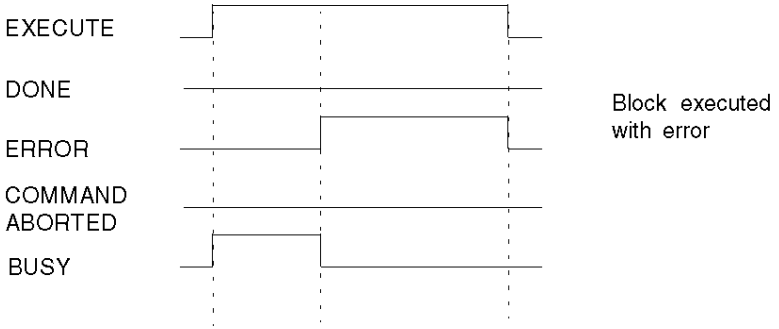
All blocks have the propriety of recording the last ERRID error in the Unity DIAG BUFFER.

Block Operation with an Execute Input Parameter.

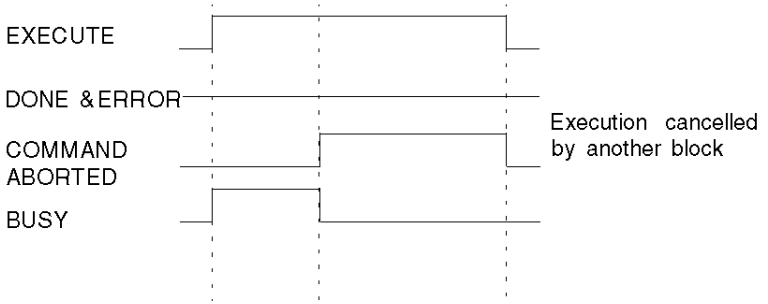
The figure below shows the time diagram of a block including an EXECUTE input parameter set to 1 until the block execution has completed (DONE=0 and BUSY=0), the block executes without any errors:



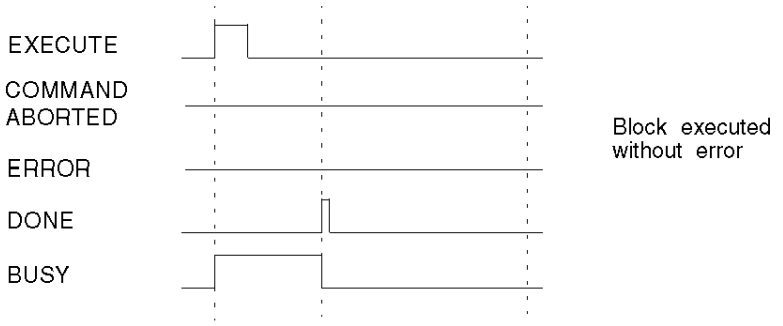
The figure below shows the time diagram of a block including an EXECUTE input parameter set to 1 until the block execution has completed (ERROR=0 and BUSY=0), the block executes with errors:



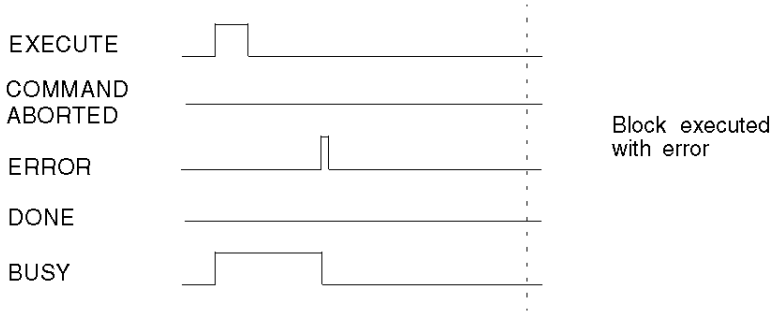
The figure below shows the time diagram of a block including an EXECUTE input parameter set to 1 until the block execution has completed (COMMANDABBORTED=0 and BUSY=0), the block execution was cancelled:



The figure below shows the time diagram of a block including an EXECUTE input parameter set to 1 on a PLC cycle, the block executes without any errors:

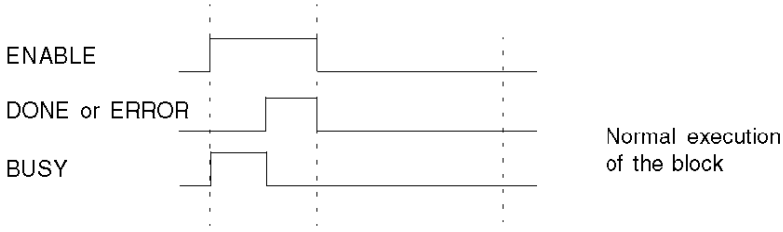


The figure below shows the time diagram of a block including an EXECUTE input parameter set to 1 on a PLC cycle, the block executes with any errors:

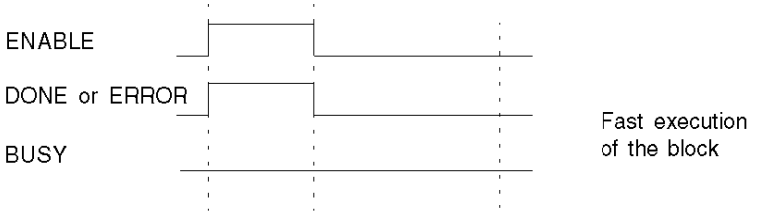


Block Operation with an Enable Input Parameter

The figure below shows the time diagram of a block including an ENABLE input parameter, execution is fast:



The figure below shows the time diagram of a block including an ENABLE input parameter, execution is normal (more than one PLC cycle):



CAN_HANDLER

Function Description

The CAN_HANDLER function is used to check the CANopen communication and that the software and physical configurations are consistent.

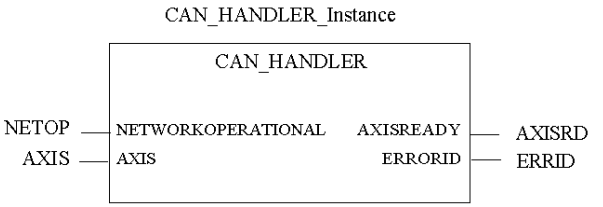
This function must be called before any call to any MFB, because it determines the correct operation of all other MFBs.

NOTE: This block must not be instantiated manually. It is automatically created when an axis is created in the Motion directory *(see Unity Pro, Operating Modes)*.

NOTE: The axis parameter must be the corresponding AXIS defined when the axis is created in the Motion directory.

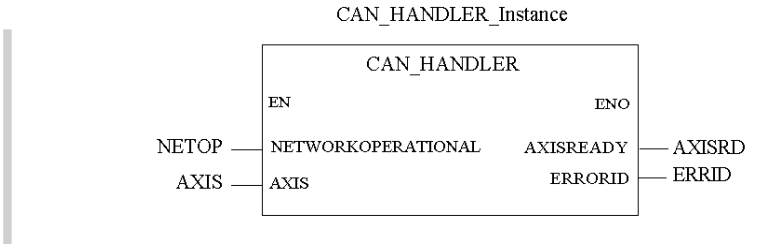
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
CAL CAN_HANDLER_Instance(NETWORKOPERATIONAL:=NETOP, AXIS:=AXIS,
AXISREADY=>AXISRD, ERRORID=>ERRID)
```

Representation in ST

Representation:

```
CAN_HANDLER_Instance(NETWORKOPERATIONAL:=NETOP, AXIS:=AXIS,
AXISREADY=>AXISRD, ERRORID=>ERRID);
```

Description of the Input Parameters

The following table describes the input parameter that is in addition to the basic parameters *(see page 33)*:

Parameter	Type	Comment
NETWORKOPERATIONAL	BOOL	Correct operation of the CANopen bus' equation result.

NOTE: Assignment of this parameter is left to the discretion of the developer. It depends on the CANopen bus management philosophy. It is recommended to assign a bit that validates the correct operation of the CANopen network.
For example, it is possible to assign the bit SLAVE_ACTIV_X (where X is the CANopen address of the device) taken from:

- the IODDT T_COM_CPP110 for Premium (**TSX CPP110** card),
- the IODDT T_COM_CO_BMX or T_COM_CO_BMX_EXPERT for Modicon M340 CANopen,

For M580 PAC, it is possible to assign a bit taken from the Modicon X80 CANopen master module device DDT *(see Modicon M580, BMECXM CANopen Modules, User Manual)*.

Description of the Output Parameters

The following table describes the output parameter that is in addition to the basic parameters *(see page 33)*:

Parameter	Type	Comment
AXISREADY	BOOL	The software configuration is consistent with the current hardware configuration and the CANopen bus is OK.

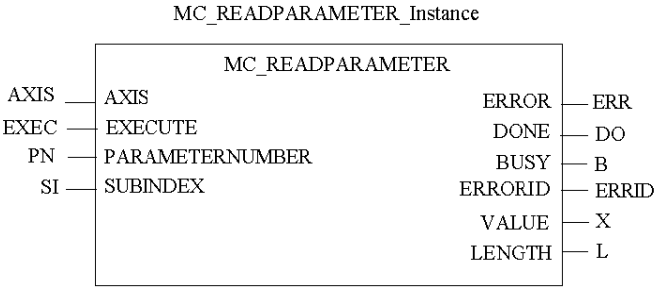
MC_READPARAMETER

Function Description

The `MC_READPARAMETER` function is used to read, via Service Data Object (SDO) messaging, a variable in the servodrive defined by the `AXIS` parameter.

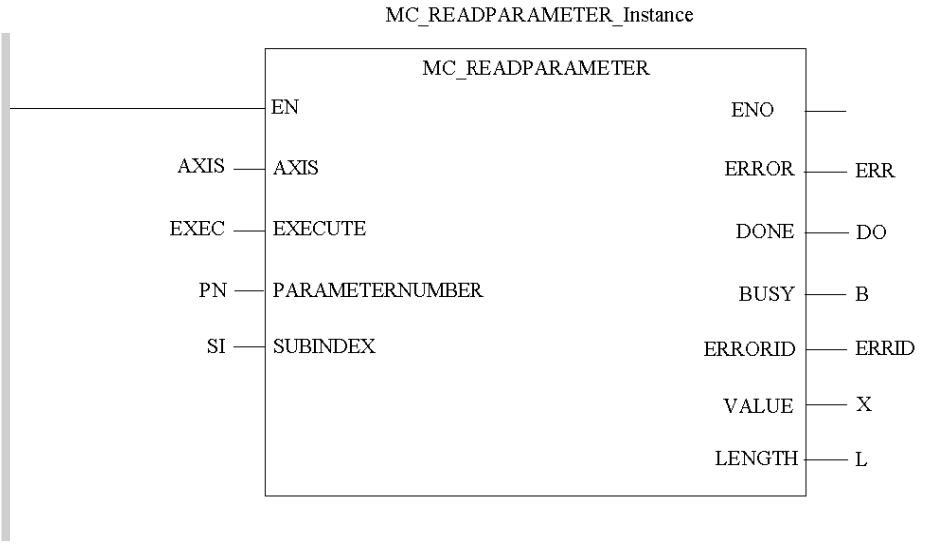
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
LD Axis
CAL MC_READPARAMETER_Instance (AXIS:=AXIS, EXECUTE:=EXEC, PARAMETERNUMB
ER:=PN, SUBINDEX:=SI, ERROR=>ERR, DONE=>DO, BUSY=>B, ERRORID=>ERRID, VA
LUE=>X, LENGTH=>L)
```

Representation in ST

Representation:

```
MC_READPARAMETER_Instance (AXIS:=AXIS, EXECUTE:=EXEC, PARAMETERNUMBER:=
PN, SUBINDEX:=SI, ERROR=>ERR, DONE=>DO, BUSY=>B, ERRORID=>ERRID, VALUE=
>X, LENGTH=>L);
```

Description of the Input Parameters

The following table describes the input parameters that are in addition to the basic *(see page 33)* parameters:

Parameter	Type	Comment
PARAMETERNUMBER	UINT	CANopen index number.
SUBINDEX	UINT	CANopen subindex number.

Description of the Output Parameters

The following table describes the output parameters that are in addition to the basic *(see page 33)* parameters:

Parameter	Type	Comment
VALUE	DINT	Value read.
LENGTH	UINT	Length in bytes of the value read.

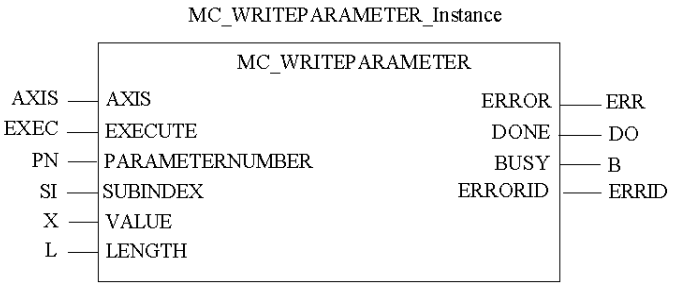
MC_WRITEPARAMETER

Function Description

The `MC_WRITEPARAMETER` function is used to write, via System Data Object (SCO) messaging, a variable in the servodrive defined by the `Axis` parameter.

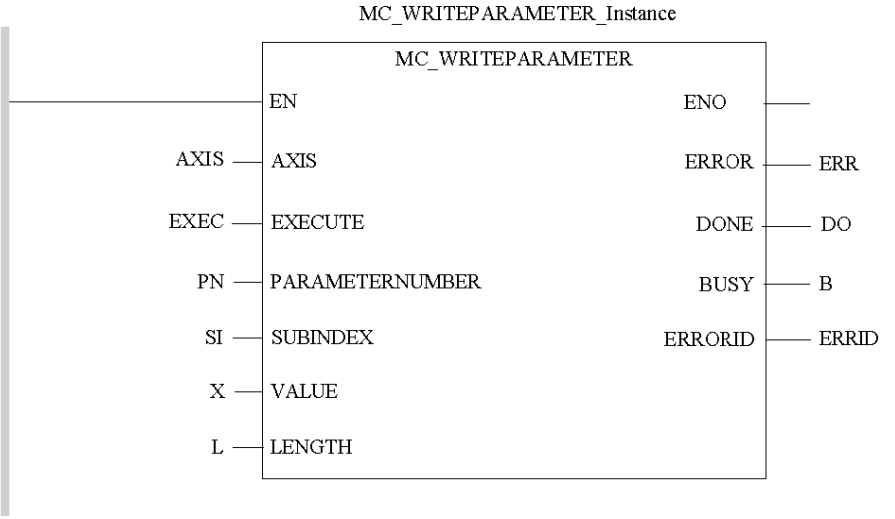
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
LD Axis
CAL MC_WRITEPARAMETER_Instance(AXIS:=AXIS, EXECUTE:=EXEC, PARAMETERNUMB
ER:=PN, SUBINDEX:=SI, VALUE:=X, LENGTH:=L, ERROR=>ERR, DONE=>DO, BUSY=>
B, ERRORID=>ERRID)
```

Representation in ST

Representation:

```
MC_WRITEPARAMETER_Instance(AXIS:=AXIS, EXECUTE:=EXEC, PARAMETERNUMBER:=
PN, SUBINDEX:=SI, VALUE:=X, LENGTH:=L, ERROR=>ERR, DONE=>DO, BUSY=>B, E
RRORID=>ERRID);
```

Description of the Input Parameters

The following table describes the input parameters that are in addition to the basic *(see page 33)* parameters:

Parameter	Type	Comment
PARAMETERNUMBER	UINT	CANopen index number to be written.
SUBINDEX	UINT	CANopen subindex number.
VALUE	DINT	Value to write.
LENGTH	UINT	Length in bytes of the value to write.

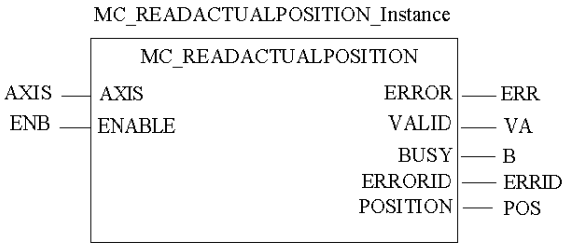
MC_READACTUALPOSITION

Function Description

The `MC_READACTUALPOSITION` function is used to read the axis position.

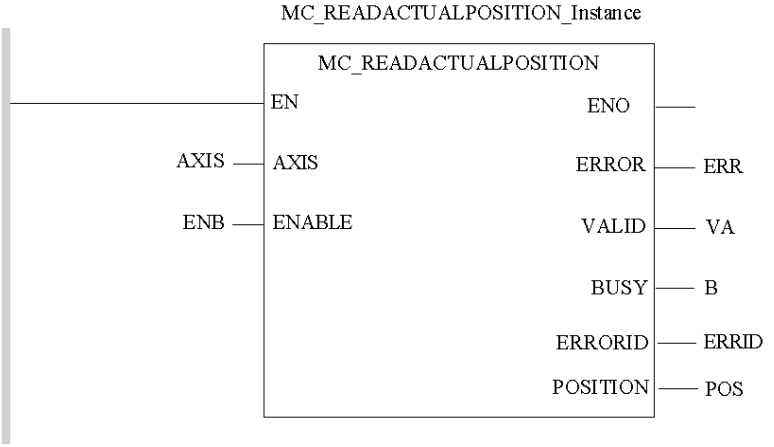
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
LD Axis
CAL MC_READACTUALPOSITION_Instance(AXIS:=AXIS, ENABLE:=ENB, ERROR=>ERR,
VALID=>VA, BUSY=>B, ERRORID=>ERRID, POSITION=>POS)
```

Representation in ST

Representation:

```
MC_READACTUALPOSITION_Instance(AXIS:=AXIS, ENABLE:=ENB, ERROR=>ERR,
VALID=>VA, BUSY=>B, ERRORID=>ERRID, POSITION=>POS);
```

Description of the Output Parameters

The following table describes the output parameter that is in addition to the basic *(see page 33)* parameters:

Parameter	Type	Comment
POSITION	DINT	Position of the axis, if VALID is TRUE.

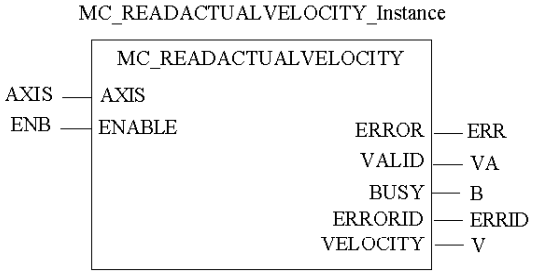
MC_READACTUALVELOCITY

Function Description

The `MC_READACTUALVELOCITY` function is used to determine the current speed of the axis.

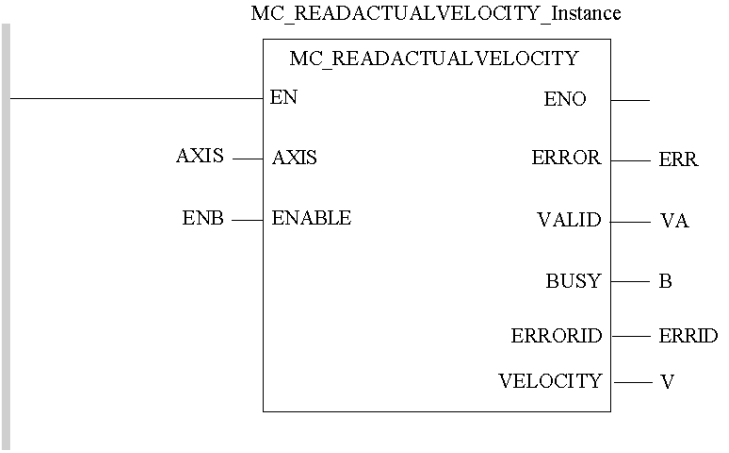
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
LD Axis
CAL MC_READACTUALVELOCITY_Instance(AXIS:=AXIS, ENABLE:=ENB, ERROR=>ERR,
 VALID=>VA, BUSY=>B, ERRORID=>ERRID, VELOCITY=>V)
```

Representation in ST

Representation:

```
MC_READACTUALVELOCITY_Instance(AXIS:=AXIS, ENABLE:=ENB, ERROR=>ERR, VAL
ID=>VA, BUSY=>B, ERRORID=>ERRID, VELOCITY=>V);
```

Description of the Output Parameters

The following table describes the output parameter that is in addition to the basic *(see page 33)* parameters:

Parameter	Type	Comment
VELOCITY	DINT	Speed of the axis, when VALID is TRUE.

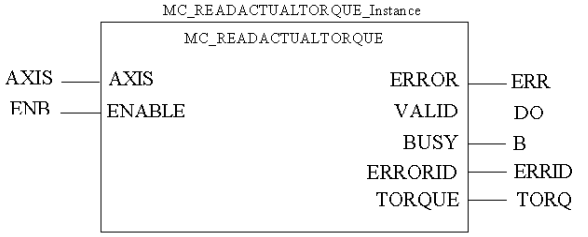
MC_READACTUALTORQUE

Function Description

The `MC_READACTUALTORQUE` function is used to return the current torque.

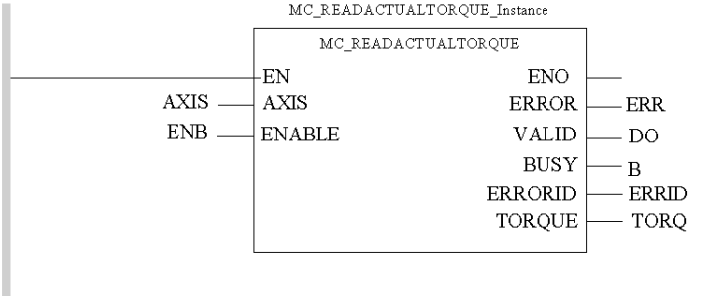
Representation in FBD

Representation



Representation in LD

Representation



Representation in IL

Representation:

```
LD Slave
CAL MC_READACTUALTORQUE_Instance(AXIS:=AXIS, ENABLE:=ENB, ERROR=>ERR, V
ALID=>DO, BUSY=>B, ERRORID=>ERRID, TORQUE=>TORQ
```

Representation in ST

Representation:

```
MC_READACTUALTORQUE_Instance(AXIS:=AXIS, ENABLE:=ENB, ERROR=>ERR, VALID
=>DO, BUSY=>B, ERRORID=>ERRID, TORQUE=>TORQ;
```

Description of the Output Parameters

The following table describes the output parameters that are in addition to the basic parameters *(see page 33)*:

Parameter	Type	Comment
TORQUE	INT	Updated every 200 ms.

The following table describes the units for each drive

Drive	15MP/HP	15LP	Lex05 / Lex32	Icla	ATV31	ATV71
Unit	1/1000 rated torque	1/100 Amp	0.01 Nominal Motor Torque	-	0.01 Nominal Torque	0.01 Nominal Torque

NOTE: The returned value for the Lexium05 and Lexium32 is a current.

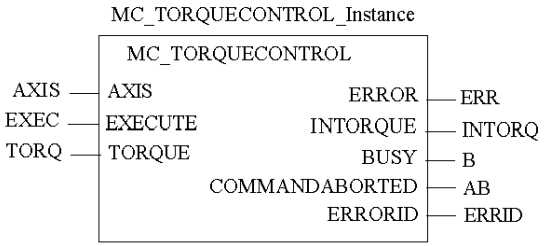
MC_TORQUECONTROL

Function Description

The `MC_TORQUECONTROL` function is used to set the drive in TORQUECONTROL mode if it is not already the case, and provide a torque command value. This function block continuously applies a torque or force, and sets the InTorque output if the torque level is reached. This function block is applicable for force and torque. When no external load is applicable, positive torque is in the positive direction of velocity.

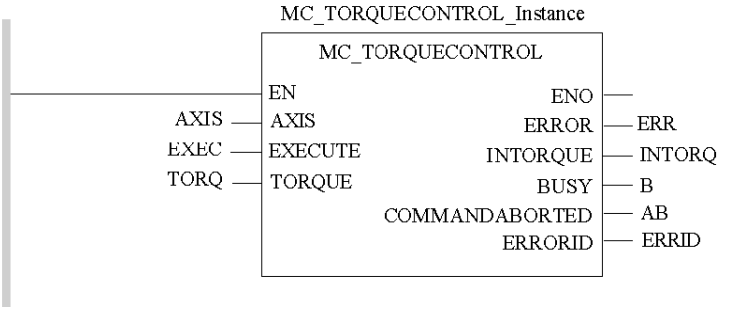
Representation in FBD

Representation



Representation in LD

Representation



Representation in IL

Representation:

```
LD Slave
CAL MC_TORQUECONTROL_Instance(AXIS:=AXIS, EXECUTE:=EXEC, TORQUE:=TORQ,
ERROR=>ERR, INTORQUE=>INTORQ, BUSY=>B, COMMANDABORTED=>AB, ERRORID=>ERR
ID)
```

Representation in ST

Representation:

```
MC_TORQUECONTROL_Instance(AXIS:=AXIS, EXECUTE:=EXEC, TORQUE:=TORQ, ERRO
R=>ERR, INTORQUE=>INTORQ, BUSY=>B, COMMANDABORTED=>AB, ERRORID=>ERRID);
```

Description of the Input Parameters

The following table describes the input parameters that are in addition to the basic parameters *(see page 33)*:

Parameter	Type	Comment
TORQUE	INT	Torque value

The following table describes units for each drive.

Drive	Lexium 15MM/ HP	Lexium 15LP	Lexium 05 / Lexium 32	ATV 71
Unit	1/1000 rated torque	1/1000 rated torque	0.01 Amp	1/1000 nominal motor torque

Description of the Output Parameters

The following table describes the output parameters that are in addition to the basic parameters *(see page 33)*:

Parameter	Type	Comment
INTORQUE	BOOL	INTORQUE is TRUE when the axis reaches the TORQUE in TorqueControl mode. Otherwise it remains set to FALSE (not significant for Lexium05).

NOTE: The ATV71 allows two ways to manage the torque mode: torque or speed regulation. Switching between these two modes is possible either by LI or by frequency level. LI is a digital or virtual input composed of the three MSB of the ControlWord (Bit15 to 13). When activated, this MFB always sets the bit 15 to 1. Depending on the drive configuration, both ways of switching is then available.

MC_RESET

Function Description

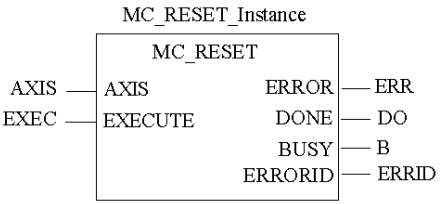
The `MC_RESET` function is used to acknowledge the internal defaults (`AxisFault`) or internal alerts (`AxisWarning`) resulting of an execution of an MFB for the axis (`Axis_ref`). The function resets the contents of the `ReadAxisError` block, which contains the diagnostic information.

In the chart status, the function allows you to change from the Errorstop state to the Standstill *(see page 29)* state.

The input and output parameters *(see page 33)* are basic parameters.

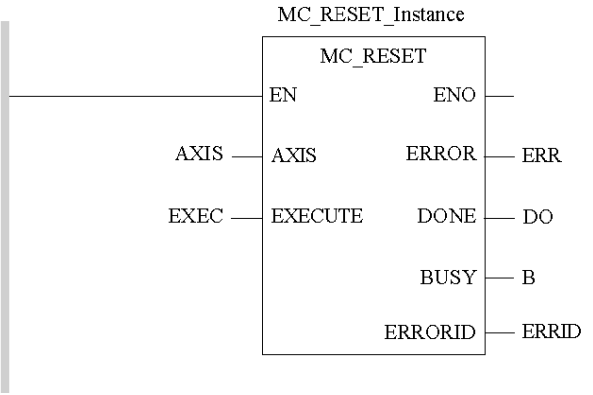
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
LD Axis
CAL MC_RESET_Instance(AXIS:=AXIS, EXECUTE:=EXEC, ERROR=>ERR, DONE=>DO,
BUSY=>B, ERRORID=>ERRID)
```

Representation in ST

Representation:

```
MC_RESET_Instance(AXIS:=AXIS, EXECUTE:=EXEC, ERROR=>ERR, DONE=>DO, BUSY
=>B, ERRORID=>ERRID);
```

MC_STOP

Function Description

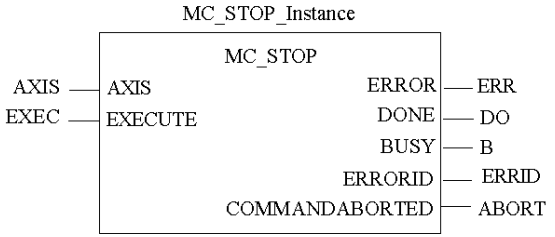
The MC_STOP function is used to stop any motion in progress and place the AXIS in Stopping status *(see page 29)*.

As soon as the axis is at zero speed, the output parameter DONE is set to TRUE, and the axis is therefore in stopping status. The AXIS moves into Standby status if the input parameter returns to FALSE.

The input and output parameters of the block are basic parameters *(see page 33)*.

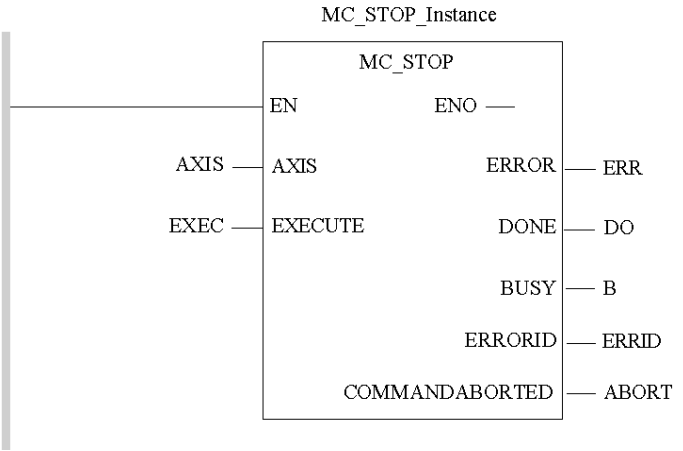
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
LD Axis
CAL MC_STOP_Instance(AXIS:=AXIS, EXECUTE:=EXEC, ERROR=>ERR, DONE=>DO, B
USY=>B, ERRORID=>ERRID, COMMANDABORTED=>ABORT)
```

Representation in ST

Representation:

```
MC_STOP_Instance(AXIS:=AXIS, EXECUTE:=EXEC, ERROR=>ERR, DONE=>DO, BUSY=
>B, ERRORID=>ERRID, COMMANDABORTED=>ABORT);
```

Stop on Z Axis

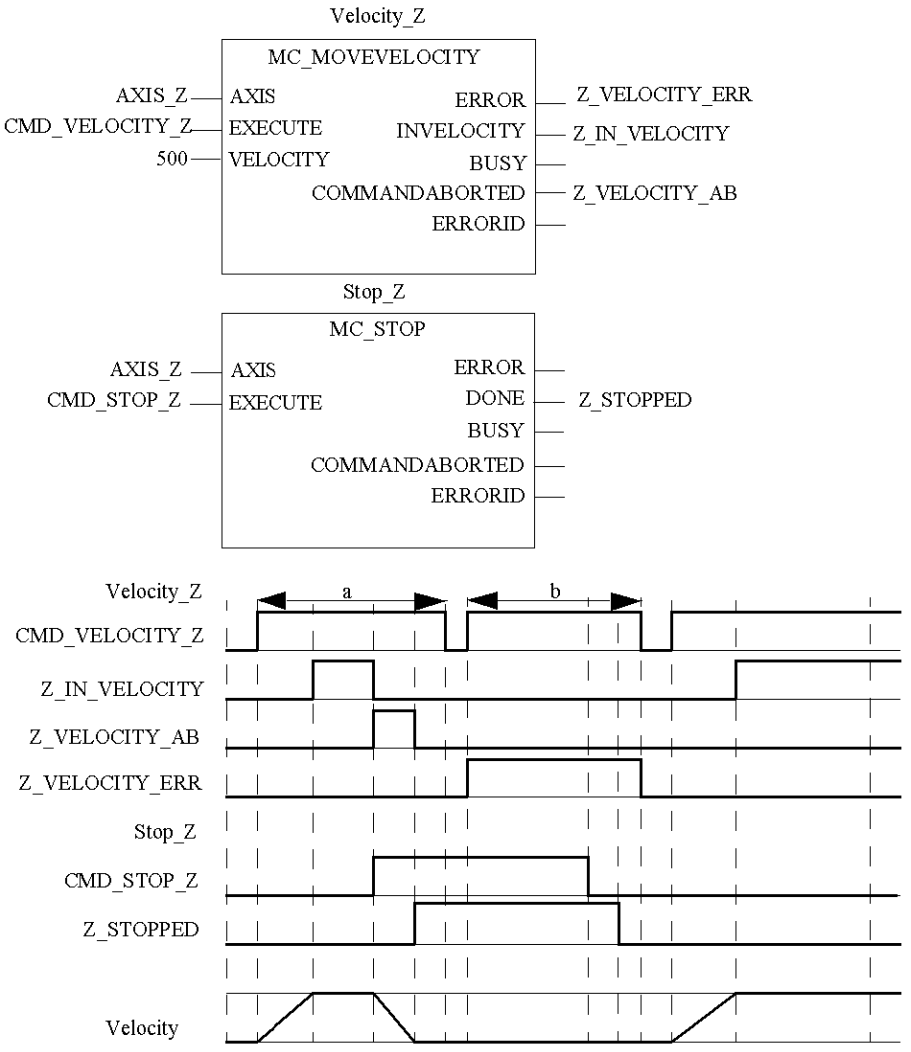
The example below shows how the `MC_STOP` behaves in combination with `MC_MOVEVELOCITY`.

The graph below shows the timing chart for the example with:

a) A rotating axis stopped with an `MC_Stop` MFB

b) The axis rejects the move command when the `Execute` parameter of the `MC_STOP` block equals 1. The `MC_MOVEVELOCITY` MFB raises an error indicating that the `MC_STOP` command is active.

The graph below shows the timing chart for stopping the axis:



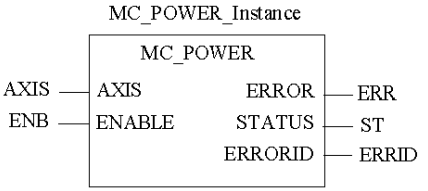
MC_POWER

Function Description

The MC_POWER function is used to move the AXIS status from Disabled *(see page 29)* to Standstill.

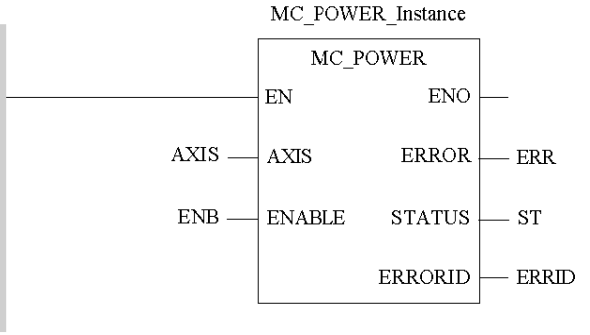
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
LD Axis
CAL MC_POWER_Instance(AXIS:=AXIS, ENABLE:=ENB, ERROR=>ERR, STATUS=>ST,
ERRORID=>ERRID)
```

Representation in ST

Representation:

```
MC_POWER_Instance(AXIS:=AXIS, ENABLE:=ENB, ERROR=>ERR, STATUS=>ST, ERRO
RID=>ERRID);
```

Description of the Output Parameters

The following table describes the output parameter that is in addition to the basic *(see page 33)* parameters:

Parameter	Type	Comment
`STATUS`	`BOOL`	If `STATUS` is set to `TRUE`, the status is Standstill *(see page 29)*. If `STATUS` is set to `FALSE`, the status is Disabled *(see page 29)*.

MC_MOVEABSOLUTE

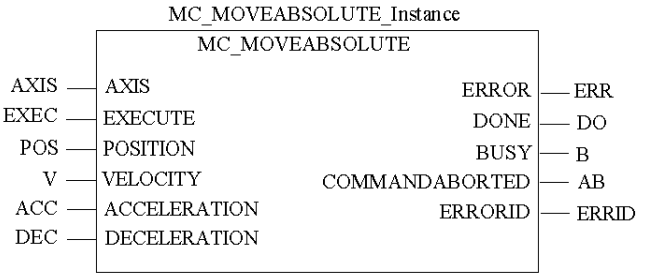
Function Description

The `MC_MOVEABSOLUTE` function is used to execute a move to absolute position command.

`Done` equals `TRUE` if the position is reached and the speed equals zero.

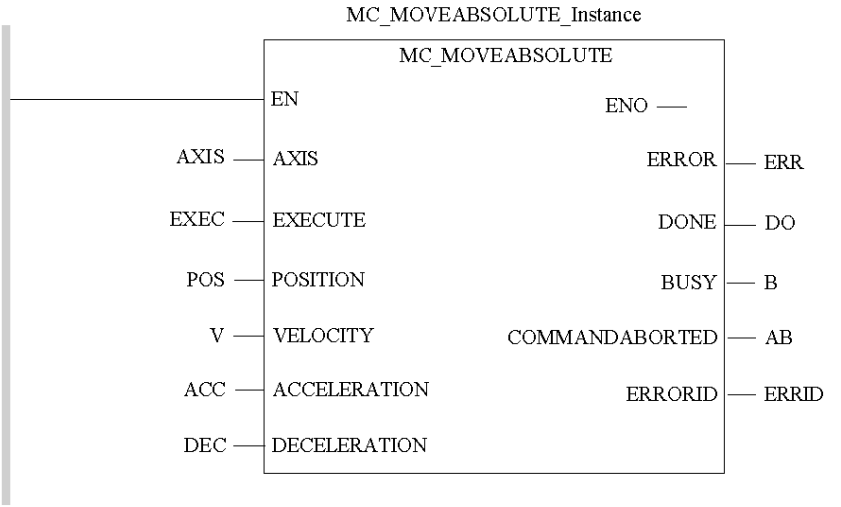
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
LD Axis
CAL MC_MOVEABSOLUTE_Instance(AXIS:=AXIS, EXECUTE:=EXEC, POSITION:=POS,
VELOCITY:=V, ACCELERATION:=ACC, DECELERATION:=DEC, ERROR=>ERR, DONE=>DO
, BUSY=>B, COMMANDABORTED=>AB, ERRORID=>ERRID)
```

Representation in ST

Representation:

```
MC_MOVEABSOLUTE_Instance(AXIS:=AXIS, EXECUTE:=EXEC, POSITION:=POS, VELO
CITY:=V, ACCELERATION:=ACC, DECELERATION:=DEC, ERROR=>ERR, DONE=>DO, BU
SY=>B, COMMANDABORTED=>AB, ERRORID=>ERRID);
```

Description of the Input Parameters

The following table describes the input parameters that are in addition to the basic *(see page 33)* parameters:

Parameter	Type	Comment
POSITION	DINT	Value specifying the position of the servodrive.
VELOCITY	UDINT	Speed value.
ACCELERATION	UDINT	Acceleration value.
DECELERATION	UDINT	Deceleration value.

NOTE: The value 0 for the acceleration and deceleration parameters allows you to use the value configured in the servodrive. If the value is other than 0, the servodrive takes this value into account.

NOTE: For the Icla, the acceleration value is the same for acceleration and deceleration. The type is `UINT`. The deceleration parameter is not active.

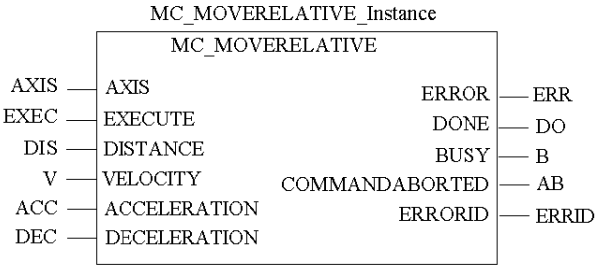
MC_MOVERELATIVE

Function Description

The MC_MOVERELATIVE function is used to execute a move to relative position command (in relation to the current position of the servodrive).

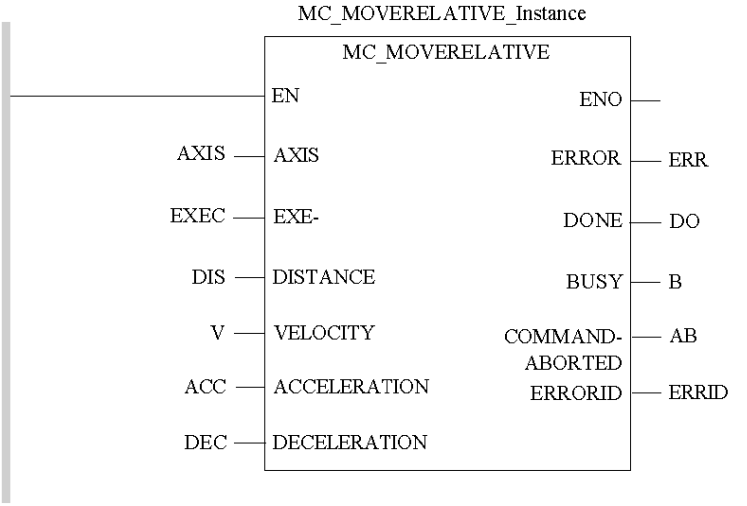
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
LD Axis
CAL MC_MOVERELATIVE_Instance(AXIS:=AXIS, EXECUTE:=EXEC, DISTANCE:=DIS,
VELOCITY:=V, ACCELERATION:=ACC, DECELERATION:=DEC, ERROR=>ERR, DONE=>DO
, BUSY=>B, COMMANDABORTED=>AB, ERRORID=>ERRID)
```

Representation in ST

Representation:

```
MC_MOVERELATIVE_Instance(AXIS:=AXIS, EXECUTE:=EXEC, DISTANCE:=DIS, VELO
CITY:=V, ACCELERATION:=ACC, DECELERATION:=DEC, ERROR=>ERR, DONE=>DO, BU
SY=>B, COMMANDABORTED=>AB, ERRORID=>ERRID);
```

Description of the Input Parameters

The following table describes the input parameters that are in addition to the basic *(see page 33)* parameters:

Parameter	Type	Comment
DISTANCE	DINT	Value specifying the distance that the servodrive must cover.
VELOCITY	UDINT	Speed value.
ACCELERATION	UDINT	Acceleration value.
DECELERATION	UDINT	Deceleration value.

NOTE: The value 0 for the acceleration and deceleration parameters allows you to use the value configured in the servodrive. If the value is other than 0, the servodrive takes this value into account.

MC_MOVEADDITIVE

Function Description

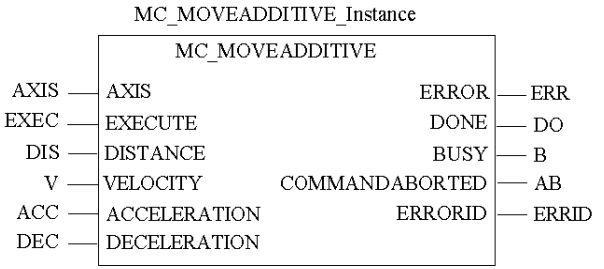
The MC_MOVEADDITIVE function is used to apply an additional move command to a servodrive that is already in motion.

As soon as the block input parameter EXECUTE is active:

- the MC_MOVEADDITIVE block takes control over the currently moving block (MC_MOVERELATIVE or MC_MOVE_ABSOLUTE),
- the value entered in the DISTANCE parameter is added to the target value of the current movement,
- the VELOCITY, ACCELERATION, and DECELERATION input parameters are taken into account immediately.

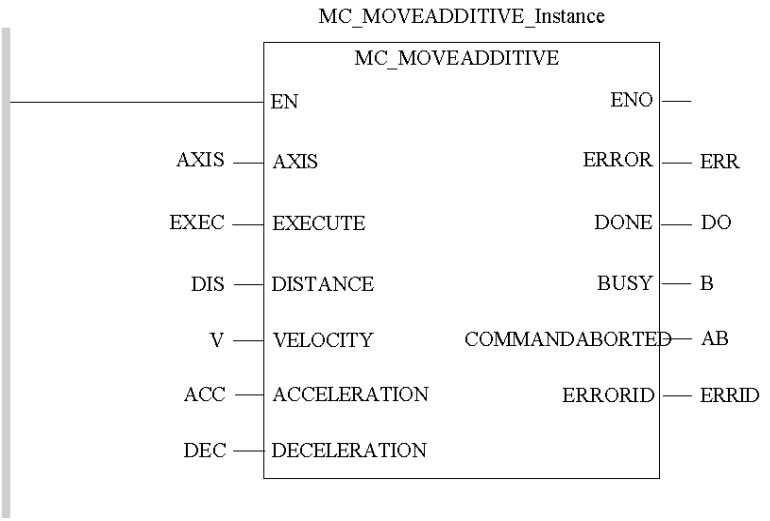
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
LD Axis
CAL MC_MOVEADDITIVE_Instance(AXIS:=AXIS, EXECUTE:=EXEC, DISTANCE:=DIS,
VELOCITY:=V, ACCELERATION:=ACC, DECELERATION:=DEC, ERROR=>ERR, DONE=>DO
, BUSY=>B, COMMANDABORTED=>AB, ERRORID=>ERRID)
```

Representation in ST

Representation:

```
MC_MOVEADDITIVE_Instance(AXIS:=AXIS, EXECUTE:=EXEC, DISTANCE:=DIS, VELO
CITY:=V, ACCELERATION:=ACC, DECELERATION:=DEC, ERROR=>ERR, DONE=>DO, BU
SY=>B, COMMANDABORTED=>AB, ERRORID=>ERRID);
```

Description of the Input Parameters

The following table describes the input parameters that are in addition to the basic *(see page 33)* parameters:

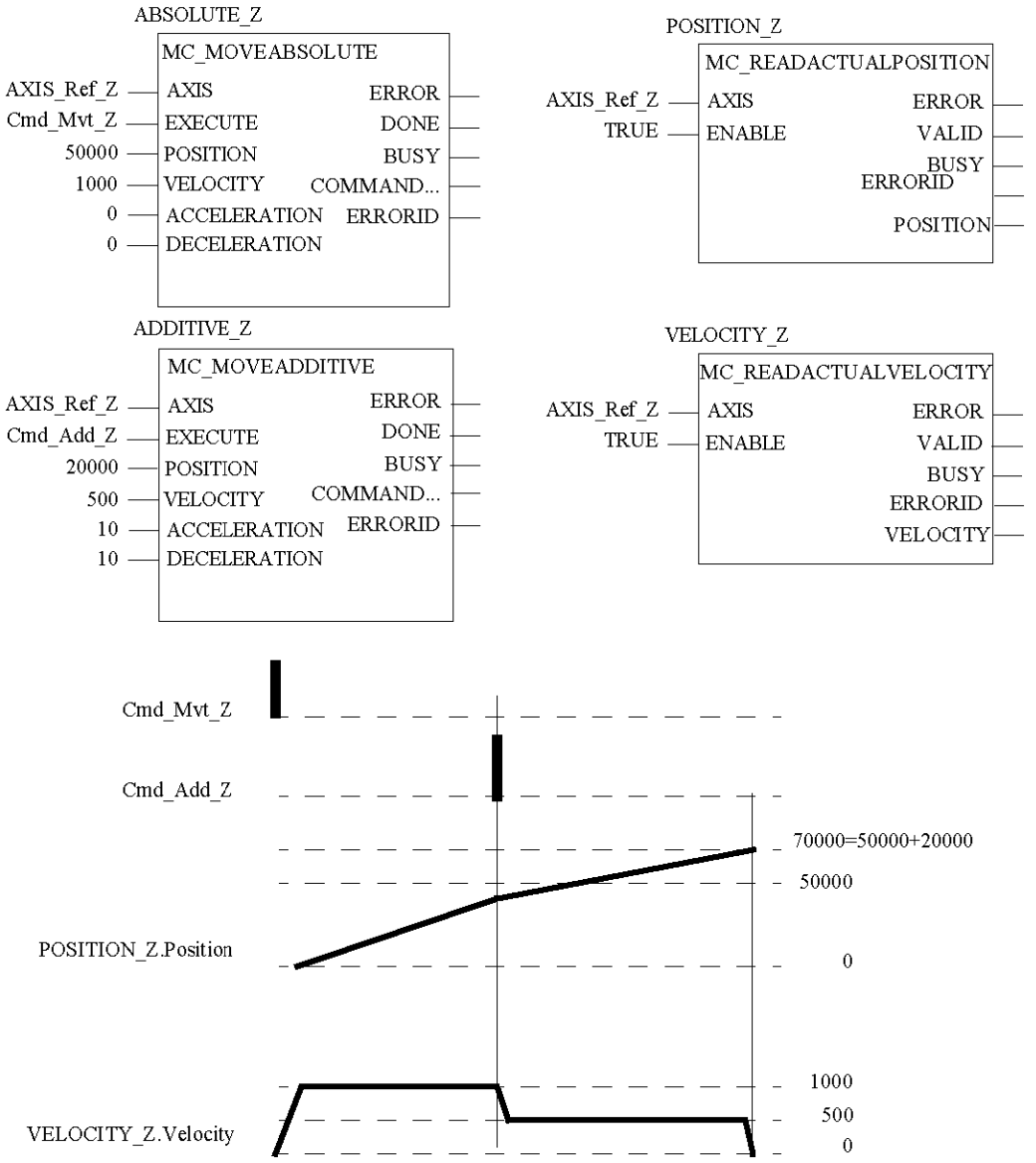
Parameter	Type	Comment
DISTANCE	DINT	Value specifying the additional distance to cover.
VELOCITY	UDINT	Speed value.
ACCELERATION	UDINT	Acceleration value.
DECELERATION	UDINT	Deceleration value.

NOTE: The value 0 for the acceleration and deceleration parameters allows you to use the value configured in the servodrive. If the value is other than 0, the servodrive takes this value into account.

NOTE: For the Icla, the acceleration value is the same for acceleration and deceleration. The type is UINT. The deceleration parameter is not active.

Example

The graph below illustrates the operation of the MC_MOVEADDITIVE block in combination with a MC_MOVE_ABSOLUTE movement block.



MC_MOVEVELOCITY

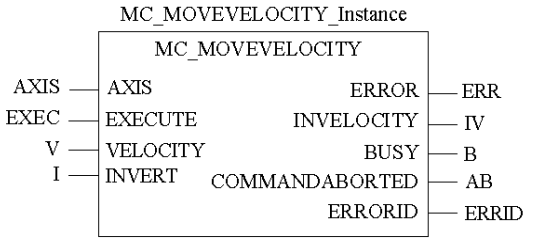
Function Description

The `MC_MOVEVELOCITY` function is used to execute an endless move command at a given speed. To use this function, the current axis state must be continuous or StandStill.

This block, in combination with the MC_STOP block *(see page 55)* can be used in manual mode.

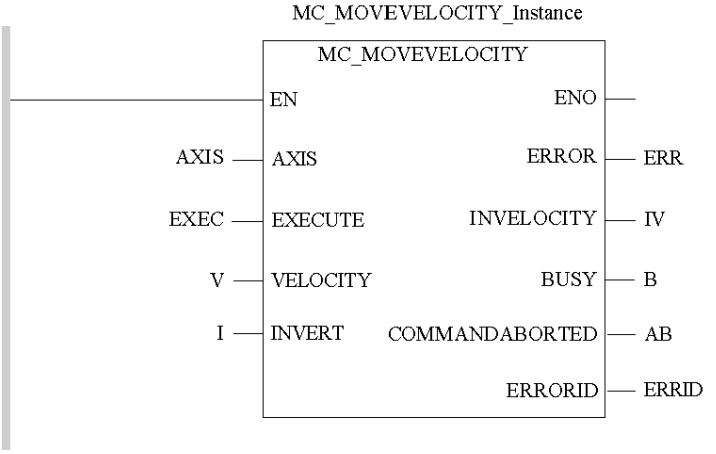
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
LD Axis
CAL MC_MOVEVELOCITY_Instance(AXIS:=AXIS, EXECUTE:=EXEC, VELOCITY:=V, IN
VERT:=I, ERROR=>ERR, INVELOCITY=>IV, BUSY=>B, COMMANDABORTED=>AB, ERROR
ID=>ERRID)
```

Representation in ST

Representation:

```
MC_MOVEVELOCITY_Instance(AXIS:=AXIS, EXECUTE:=EXEC, VELOCITY:=V, INVERT
:=I, ERROR=>ERR, INVELOCITY=>IV, BUSY=>B, COMMANDABORTED=>AB, ERRORID=>
ERRID);
```

Description of the Input Parameters

The following table describes the input parameters that are in addition to the basic parameters *(see page 33)*:

Parameter	Type	Comment
VELOCITY	DINT	Speed value. For ICLA or ATV31 servodrives, the VELOCITY parameter is used as INT type.
INVERT	Bool	Direction of rotation. If INVERT is set to TRUE, the direction is the inverse of the VELOCITY sign. When INVERT is set to FALSE, the direction is that of the VELOCITY.

Description of the Output Parameters

The following table describes the output parameters that are in addition to the basic parameters *(see page 33)*:

Parameter	Type	Comment
INVELOCITY	BOOL	INVELOCITY is TRUE when the VELOCITY speed is reached; otherwise it remains set to FALSE.

MC_JOG

Function Description

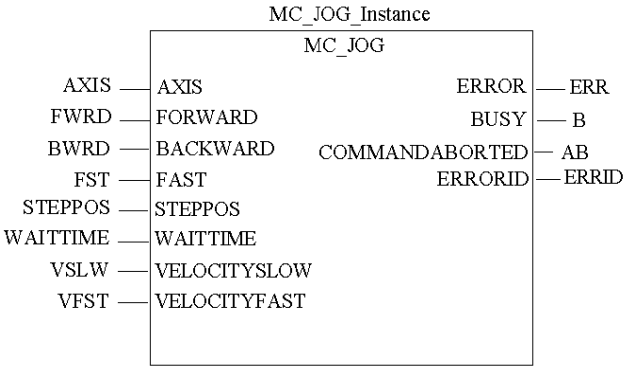
The `MC_JOG` function is used to set the drive in Position mode if it is not already the case, and provide a direction and a speed command value. This function is accepted only if the State diagram is in **STANDSTILL** state.

This function allows:

- Moving by position step or continuously (Lexium05, Lexium32, and Icla),
- Moving out of the `LimitSwitch` (Lexium05 and Icla) in case of `LimitSwitch` detected error situation:
  - When starting the movement in the appropriate direction, the axis state is commuted into homing state.
  - In case of release of `MC_Home` before being out of the LimitSwitch, an `errorID` 8 appears. A `MC_Reset` command is necessary before doing another `MC_JOG` function.

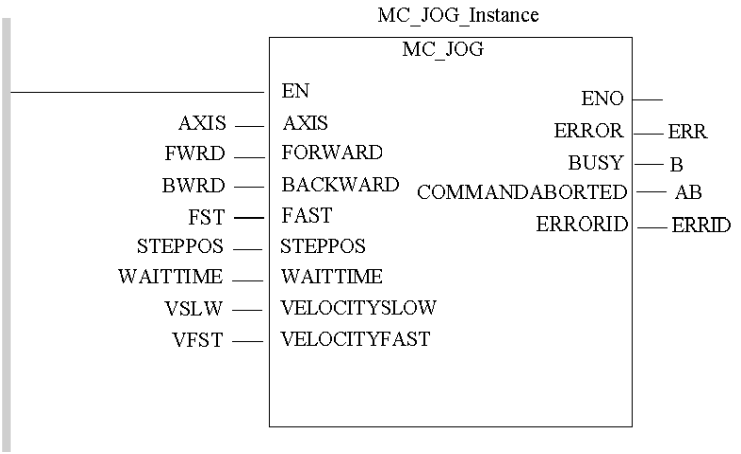
Representation in FBD

Representation



Representation in LD

Representation



Representation in IL

Representation:

```
LD Slave
CAL MC_JOG_Instance(AXIS:=AXIS FORWARD:=FWRD, BACKWARD:=BWRD, FAST:=FST
, STEPPOS:=STEPPOS, WAITTIME:=WAITTIME, VELOCITYSLOW:=VSLW, VELOCITYFAS
T:=VFST, ERROR=>ERR, BUSY=>B, COMMANDABORTED=>AB, ERRORID=>ERRID)
```

Representation in ST

Representation:

```
MC_JOG_Instance(AXIS:=AXIS FORWARD:=FWRD, BACKWARD:=BWRD, FAST:=FST, ST
EPPOS:=STEPPOS, WAITTIME:=WAITTIME, VELOCITYSLOW:=VSLW, VELOCITYFAST:=V
FST, ERROR=>ERR, BUSY=>B, COMMANDABORTED=>AB, ERRORID=>ERRID));
```

Description of the Input Parameters

The following table describes the input parameters that are in addition to the basic parameters *(see page 33)*:

Parameter	Type	Comment
FORWARD	BOOL	Forward operating direction.
BACKWARD	BOOL	Backward operating direction.
FAST	BOOL	Jog speed selection.

Parameter	Type	Comment
STEPPOS	INT	Jogging path until continuous running.
WAITTIME	INT	Waiting period until continuous running.
VELOCITYSLOW	UDINT	Speed for slow movement. Must always be connected.
VELOCITYFAST	UDINT	Speed for fast movement. Must always be connected.

For Lexium15MP and Lexium 15HP, STEPPOS and WAITTIME inputs are not significant.

When both inputs are active, the block detects the mismatch and sets the value 10 as error code: Bad condition for Jog Direction *(see page 113)*.

For Lexium05 and Lexium32 STEPPOS and WAITTIME inputs:

- Must be set,
- Are sent by SDO,
- Are not sent if the value is equal to -1. It signifies that update of this parameter is not required, and the MFB works with the actual value of these parameters already inside the drive.

The input FAST allows selecting between VELOCITYSLOW and VELOCITYFAST.

Function Behavior

Both inputs FORWARD/BACKWARD are edge active:

- Rising edge makes the movement start,
- Falling edge makes the movement stop.

Parameter Description for the Lexium 05

Group.Name Index:Subindex dec. (hex.)	Meaning Bit assignment	Units Range ?
FASTSPEED NFST JOG-nFSt	Speed for fast manual movement(8-16) The adjustment value is limited in internal on the actual parameter adjustment in RAMPn_max.	tr/min 1 180 13200
SLOWSPEED NSLW JOG-nSLL	Speed for slow manual movement(8-16) The adjustment value is limited in internal on the actual parameter adjustment in RAMPn_max.	tr/min 1 60 13200
STEPPOS	Jogging path with manual start before the continuous running 0: direct activation of continuous running >0: distance positioning by manual cycle	usr 0 20

Group.Name Index:Subindex dec. (hex.)	Meaning Bit assignment	Units Range ?
WAITTIME	Waiting period until continuous running(8-16) Only effective if jogging path is set not equal to 0	ms 1 500 32767

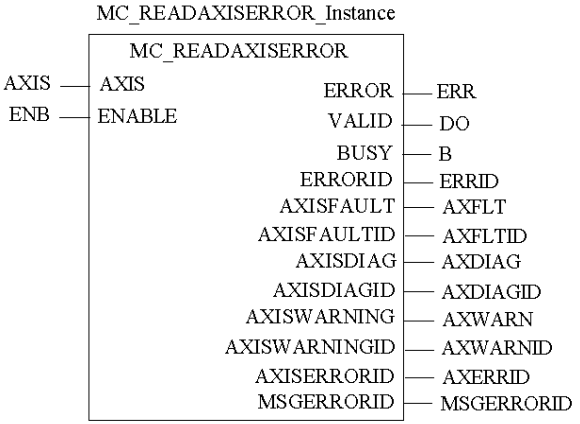
MC_READAXISERROR

Function Description

The MC_READAXISERROR function is used to recover system errors.

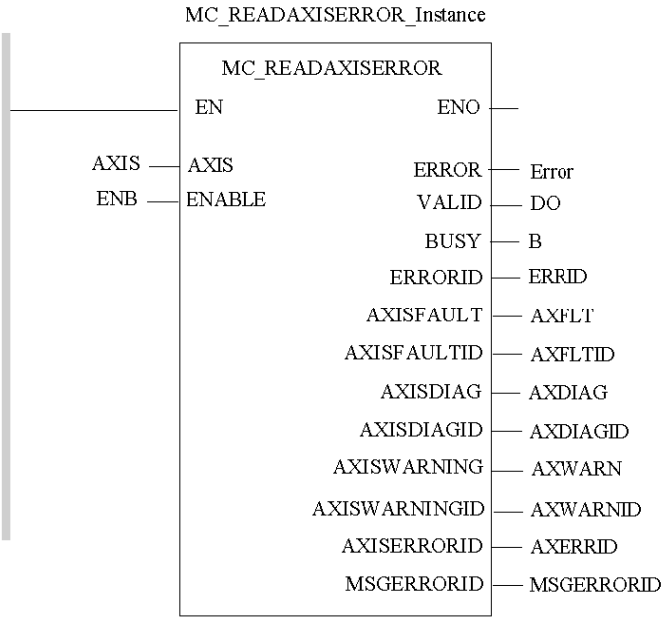
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
LD Axis
CAL MC_READAXISERROR_Instance(AXIS:=AXIS, ENABLE:=ENB, ERROR=>ERR, VALI
D=>DO, BUSY=>B, ERRORID=>ERRID, AXISFAULT=>AXFLT, AXISFAULTID=>AXFLTID,
 AXISDIAG=>AXDIAG, AXISDIAGID=>AXDIAGID, AXISWARNING=>AXWARN, AXISWARNI
NGID=>AXWARNID, AXISERRORID=>AXERRID, MSGERRORID=>MSGERRORID)
```

Representation in ST

Representation:

```
MC_READAXISERROR_Instance(AXIS:=AXIS, ENABLE:=ENB, ERROR=>ERR, VALID=>D
O, BUSY=>B, ERRORID=>ERRID, AXISFAULT=>AXFLT, AXISFAULTID=>AXFLTID, AXI
SDIAG=>AXDIAG, AXISDIAGID=>AXDIAGID, AXISWARNING=>AXWARN, AXISWARNINGID
=>AXWARNID, AXISERRORID=>AXERRID, MSGERRORID=>MSGERRORID);
```

Description of the Output Parameters

The following table describes the output parameters that are in addition to the basic *(see page 33)* parameters:

Parameter	Type	Comment
AXISFAULT	BOOL	Drive error.
AXISFAULTID	UDINT	Drive error value.
AXISDIAG	BOOL	• Premium: Copy of the CPP110 diagnostic bit for the slave. • M340: Copy of the SLAVE_EMCY bit for the slave. • M580: Copy of the SLAVE_LIVE bit for the slave.
AXISDIAGID	UINT	AxisDiag value.
AXISWARNING	BOOL	Drive warning.
AXISWARNINGID	UINT	Drive warning value.
AXISERRORID	WORD	MFB error number latching.
MSGERRORID	UINT	Error latching by messaging.

Error Codes

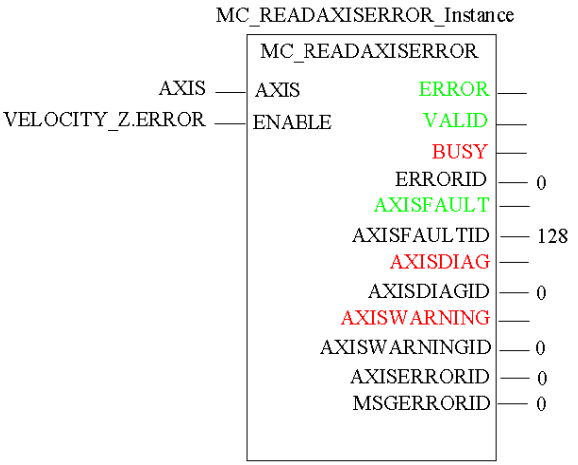
The following table shows the list of diagnostic objects and the corresponding values according to the drive types:

	ATV31	ATV71	Lexium 15MP/HP	Lexium 05 / Lexium 32	Icla	Lexium 15LP
AXISFAULTID	LFT 2029:16	LFT 2029:16	ERRCODE 2070:16	SignLatched 301C:8	FaulSig_SR 301C:12	ERRCODE 385D:01
AXISWARNINGID	-	LRS6 2002:38	1002:00	WarnLatched 301C:0C	WarnSig 301C:A	1002:00
AXISDIAGID	ERRD 603F:0	ERRD 603F:0	1003:01	StopFault 603F:0	StopFault 3020:7	1003:01

NOTE: Refer to the CANopen device documentation to identify the meaning of the code. Special feature: The Lexium 15MP ERRCODE refers to the ASCII object documentation of the Lexium.

Diagnostic

The table below describes a procedure to diagnose the behavior following the execution of an MFB. The Lexium 15MP is the device used in this example.

Step	Action
1	The AXISFAULT output parameter equals 1. The AXISFAULTID output parameter displays an error code value. The graph below shows how the diagnostic is given in this example through the code values: 
2	Refer to the ASCII object documentation of the Lexium 15MP, and look for the ASCII ERRCODE 2070:16.
3	In the ASCII documentation, the value 16#0080 (128 in decimal) for ERRCODE designates an overspeed.
4	Correct the speed constants.
5	Execute the MC_RESET block

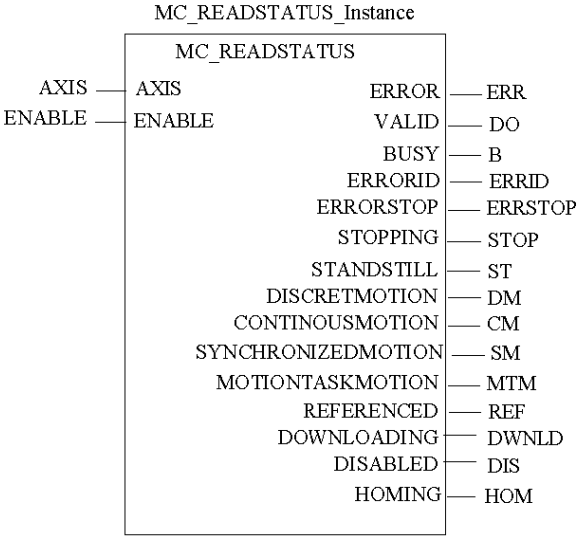
MC_READSTATUS

Function Description

The `MC_READSTATUS` function is used to determine the logical status of the servodrive *(see page 29)*.

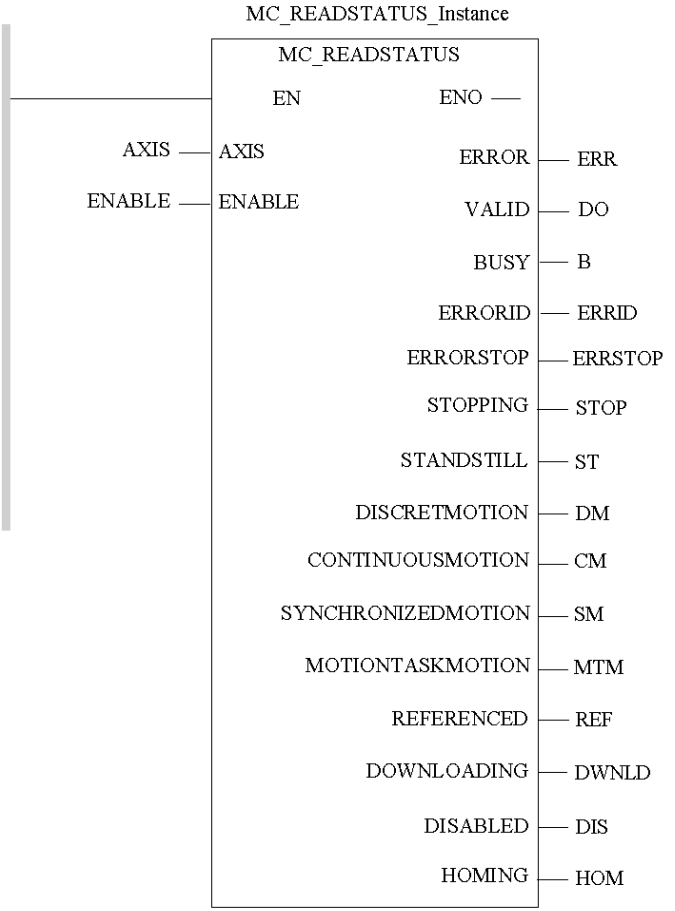
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
LD Axis
CAL MC_READSTATUS_Instance(AXIS:=AXIS, ENABLE:=ENABLE, ERROR=>ERR, VALI
D=>DO, BUSY=>B, ERRORID=>ERRID, ERRORSTOP=>ERRSTOP, STOPPING=>STOP, STA
NDSTILL=>ST, DISCRETMOTION=>DM, CONTINUOUSMOTION=>CM, SYNCHRONIZEDMOTIO
N=>SM, MOTIONTASKMOTION=>MTM, REFERENCED=>REF, DOWNLOADING=>DWNLD, DISA
BLED=>DIS, HOMING=>HOM)
```

Representation in ST

Representation:

```
MC_READSTATUS_Instance(AXIS:=AXIS, ENABLE:=ENABLE, ERROR=>ERR, VALID=>D
O, BUSY=>B, ERRORID=>ERRID, ERRORSTOP=>ERRSTOP, STOPPING=>STOP, STANDST
ILL=>ST, DISCRETMOTION=>DM, CONTINUOUSMOTION=>CM, SYNCHRONIZEDMOTION=>S
M, MOTIONTASKMOTION=>MTM, REFERENCED=>REF, DOWNLOADING=>DWNLD, DISABLED
=>DIS, HOMING=>HOM);
```

Description of the Output Parameters

The following table describes the output parameters that are in addition to the basic *(see page 33)* parameters:

Parameter	Type	Comment
ERROR	BOOL	When ERROR is TRUE, an execution error is detected by the function block.
ERRORID	INT	Axis is missing or faulty.
ERRORSTOP	BOOL	When ERRORSTOP is TRUE, stop with an error.
STOPPING	BOOL	When STOPPING is TRUE, MC_STOP is in progress.
STANDSTILL	BOOL	When StandStill is TRUE, the servodrive is in StandStill status.
DSICRETMOTION	BOOL	When DISCRETMOTION is TRUE, a move in absolute or relative mode is in progress, or Jog.
CONTINUOUSMOTION	BOOL	When CONTINUOUSMOTION is TRUE, a move in speed mode is in progress, or torque.
SYNCHRONIZEDMOTION	BOOL	A synchronization operation is in progress, via the gearing operating mode.
MOTIONTASKMOTION	BOOL	An MotionTask operation is in progress via the LXM_STARTMTASK block.
REFERENCED	BOOL	The servodrive is referenced (homing performed). **Note**: This bit is not taken into account for ATV type servodrives.
DOWNLOADING	BOOL	When DOWNLOADING is TRUE, downloading is in progress.
DISABLED	BOOL	Axis is in disable status.

Parameter	Type	Comment
HOMING	BOOL	When HOMING is TRUE: • MC_HOME is in progress • MC_JOG going out of the LimitSwitches is in progress for Lexium 05 / Icla.

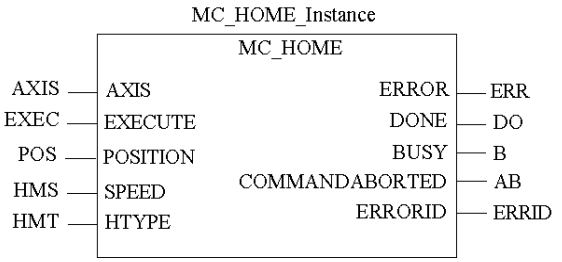
MC_HOME

Function Description

The MC_HOME function is used to execute a homing command.

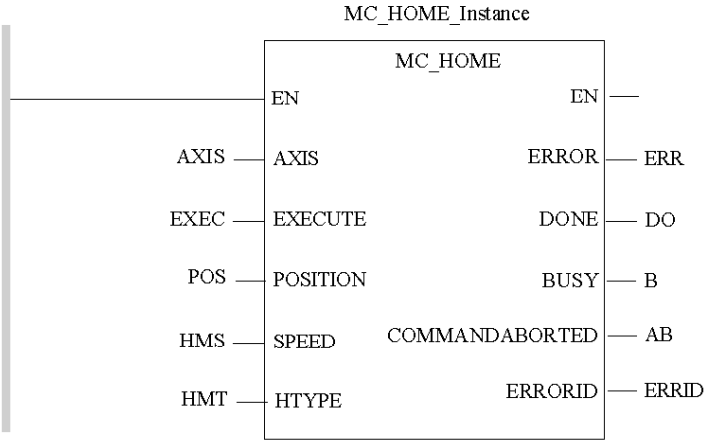
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
LD Axis
CAL MC_HOME_Instance(AXIS:=AXIS, EXECUTE:=EXEC, POSITION:=POS, SPEED:=H
MS, HTYPE:=HMT, ERROR=>ERR, DONE=>DO, BUSY=>B, COMMANDABORTED=>AB, ERRO
RID=>ERRID)
```

Representation in ST

Representation:

```
MC_HOME_Instance(AXIS:=AXIS, EXECUTE:=EXEC, POSITION:=POS, SPEED:=HMS,
HTYPE:=HMT, ERROR=>ERR, DONE=>DO, BUSY=>B, COMMANDABORTED=>AB, ERRORID=
>ERRID);
```

Description of the Input Parameters

The following table describes the input parameters that are in addition to the basic *(see page 33)* parameters:

Parameter	Type	Comment
HTYPE	INT	Homing type. For Lexium 15MP. HTYPE: if the value is set to -1, no change, the parameter is not transmitted.
POSITION	DINT	Position at the end of the homing.
SPEED	DINT	Homing speed. If the value is of 0, no change.

NOTE: The value of the homing type is described in the manual of the device to be implemented. For the Lexium 15, refer to the Homing in Lexium CANopen communication.

LXM_GEARPOS

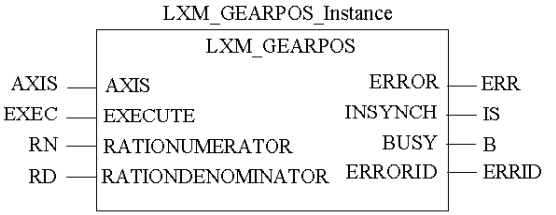
Function Description

The LXM_GEARPOS function is used to synchronize the position of a slave axis with a master axis, according to a coefficient (ratio).

NOTE: A physical connection to the auxiliary encoder command interface connector (X5) between the two servodrives must be present.

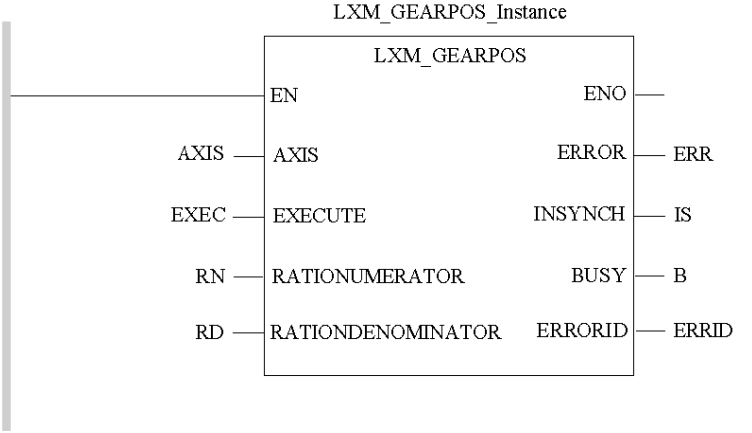
Representation in FBD

Representation



Representation in LD

Representation



Representation in IL

Representation:

```
LD Slave
CAL LXM_GEARPOS_Instance(AXIS:=AXIS, EXECUTE:=EXEC, RATIONUMERATOR:=RN,
 RATIONDENOMINATOR:=RD, ERROR=>ERR, INSYNCH=>IS, BUSY=>B, ERRORID=>ERRI
D)
```

Representation in ST

Representation:

```
LXM_GEARPOS_Instance(AXIS:=AXIS, EXECUTE:=EXEC, RATIONUMERATOR:=RN, RAT
IONDENOMINATOR:=RD, ERROR=>ERR, INSYNCH=>IS, BUSY=>B, ERRORID=>ERRID);
```

Description of the Input Parameters

The following table describes the input parameters that are in addition to the basic *(see page 33)* parameters:

Parameter	Type	Comment
AXIS	AXIS_REF	Axis_Ref *(see page 28)* type object, which defines the slave device.
RATIONUMERATOR	UINT	Motion synchronization ratio numerator (allowed range 1-32767 for LXM05).
RATIONDENOMINATOR	UINT	Motion synchronization ratio denominator.

Calcul of slave position for Lexium15MP

$$\Delta \text{position slave} = \Delta \text{position master x ratio numerator/ratio denominator}$$

Calcul of slave position for Lexium15LP

$$\Delta \text{position slave} = \Delta \text{position master x 1024 x ratio numerator/ratio denominator}$$

Description of the Output Parameters

The following table describes the output parameter that is in addition to the basic *(see page 33)* parameters:

Parameter	Type	Comment
INSYNCH	BOOL	INSYNCH is TRUE when the synchronization position, in relation to the Master, is reached. It remains TRUE as long as EXECUTE is TRUE even if the axis has stopped (not significant for Lexium 05/32).

Description Parameter for the Lexium 05 and Lexium 32

The following table describes the additional parameter for the Lexium 05 and Lexium 32:

Parameter	Type	Comment
SYNCMODE	BOOL	SYNCMODE is an additional parameter for the Lexium 05/32.The access to this parameter by MC_READPARAMETER or MC_WRITEPARAMETER before calling MFB_Gear, is possible by using: Index =0, subindex=200 as object number for SYNCMODE. FALSE: immediate synchronization: the positioning controller follows reference pulses from the time at which the gear processing is activated. TRUE: synchronization with compensation movement. With activation of the gear processing, the motor endeavors to retrieve the reference pulses accumulated.

LXM_GearPosS

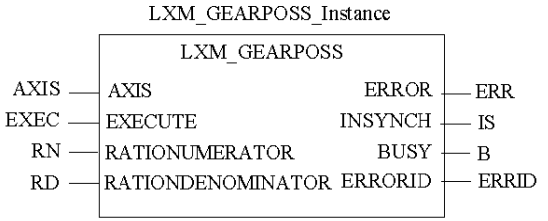
Function Description

Similar to LXM_GEARPOS, the function LXM_GEARPOSS is also used to synchronize the position of a slave axis with a master axis, according to a coefficient (ratio). It allows you to use and obtain negative values for the coefficient ratio.

NOTE: A physical connection to the auxiliary encoder command interface connector (X5) between the two servodrives must be present.

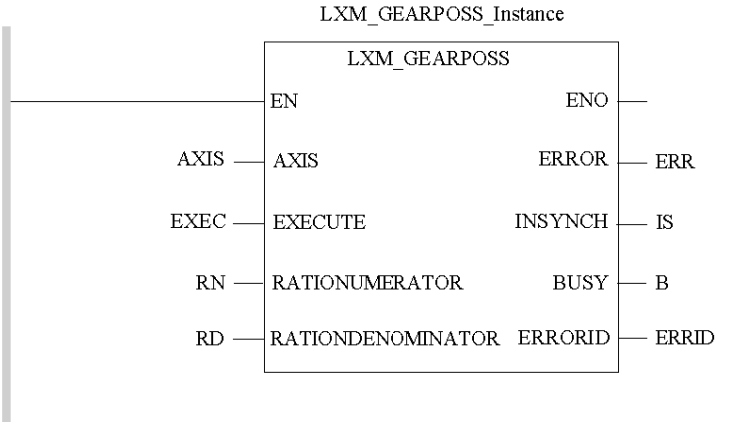
Representation in FBD

Representation



Representation in LD

Representation



Representation in IL

Representation:

```
LD Slave
CAL LXM_GEARPOSS_Instance(AXIS:=AXIS, EXECUTE:=EXEC, RATIONUMERATOR=RN,
 RATIONDENOMINATOR:=RD, ERROR=>ERR, INSYNCH=>IS, BUSY=>B, ERRORID=>ERRI
D)
```

Representation in ST

Representation:

```
LXM_GEARPOSS_Instance(AXIS:=AXIS, EXECUTE:=EXEC, RATIONUMERATOR=RN, RAT
IONDENOMINATOR:=RD, ERROR=>ERR, INSYNCH=>IS, BUSY=>B, ERRORID=>ERRID);
```

Description of the Input Parameters

The following table describes the input parameters that are in addition to the basic *(see page 33)* parameters:

Parameter	Type	Comment
AXIS	AXIS_REF	Axis_Ref *(see page 28)* type object, which defines the slave device.
RATIONUMERATOR	INT	Motion synchronization ratio numerator.
RATIONDENOMINATOR	INT	Motion synchronization ratio denominator.

Allowed values	Lexium 15MP/HP	Lexium 05/32	Lexium 15LP
Range for RATIONUMERATOR	-32768,32767	-32768,32767.	1,32767.
Range for RATIONDENOMINATOR	1,32767	1,32767.	-32768,32767.

For Lexium 15LP/MP

- The value 0 means that ”the value does not change” and so the parameter inside the drive is not updated
- When one of inputs values is out of range, the block goes on error with the already existing error code value 8: AXIS_COMMAND_REFUSED

Calcul of slave position for Lexium15MP

$$\Delta \text{ position slave} = \Delta \text{ position master x ratio numerator/ratio denominator}$$

Calcul of slave position for Lexium15LP

$$\Delta \text{ position slave} = \Delta \text{ position master x } 1024 \text{ x ratio numerator/ratio denominator}$$

Description of the Output Parameters

The following table describes the output parameter that is in addition to the basic *(see page 33)* parameters:

Parameter	Type	Comment
INSYNCH	BOOL	INSYNCH is TRUE when the synchronization position, in relation to the Master, is reached. It remains TRUE as long as EXECUTE is TRUE even if the axis has stopped (not significant for Lexium 05/32).

Description Parameter for the Lexium 05 and Lexium 32

The following table describes the additional parameter for the Lexium 05 and Lexium 32:

Parameter	Type	Comment
SYNCMODE	BOOL	SYNCMODE is an additional parameter for the Lexium 05/32.The access to this parameter by MC_READPARAMETER or MC_WRITEPARAMETER before calling MFB_Gear, is possible by using: Index =0, subindex=200 as object number for SYNCMODE. FALSE: immediate synchronization: the positioning controller follows reference pulses from the time at which the gear processing is activated. TRUE: synchronization with compensation movement. With activation of the gear processing, the motor endeavors to retrieve the reference pulses accumulated.

TE_UPLOADDRIVEPARAM

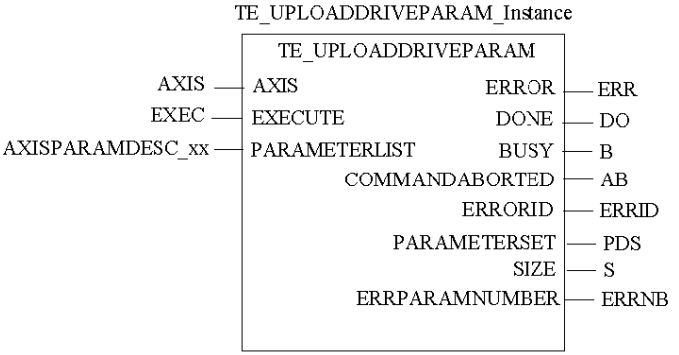
Function Description

The `TE_UPLOADDRIVEPARAM` function is used to save the parameters of one servodrive to a memory zone of a PLC.

NOTE: Before executing this function, the servodrive status must be Disabled *(see page 29)*.

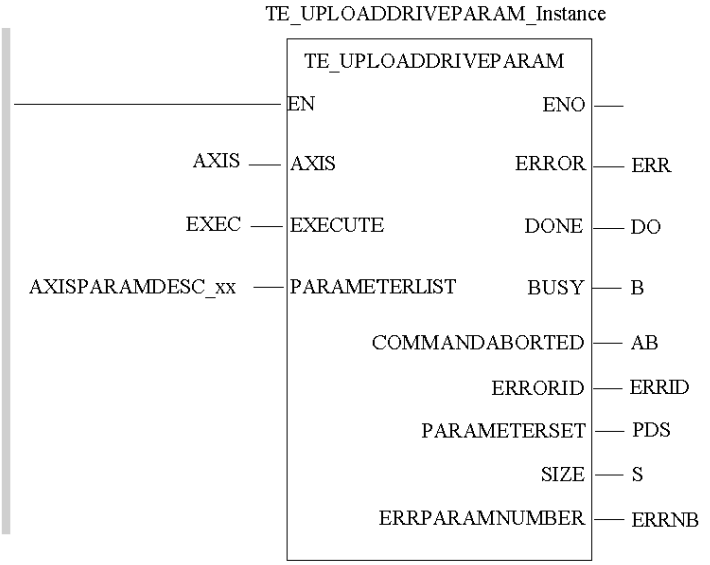
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
LD Axis
CAL TE_UPLOADDRIVEPARAM_Instance(AXIS:=AXIS, EXECUTE:=EXEC, PARAMETERLI
ST:=AXISPARAMDESC_xx, ERROR=>ERR, DONE=>DO, BUSY=>B, COMMANDABORTED=>AB
, ERRORID=>ERRID, PARAMETERSET=>PDS, SIZE=>S, ERRPARAMNUMBER=>ERRNB)
```

Representation in ST

Representation:

```
CAL TE_UPLOADDRIVEPARAM_Instance(AXIS:=AXIS, EXECUTE:=EXEC, PARAMETERLI
ST:=AXISPARAMDESC_xx, ERROR=>ERR, DONE=>DO, BUSY=>B, COMMANDABORTED=>AB
, ERRORID=>ERRID, PARAMETERSET=>PDS, SIZE=>S, ERRPARAMNUMBER=>ERRNB);
```

Description of the Input Parameters

The following table describes the input parameter that is in addition to the basic *(see page 33)* parameters:

Parameter	Type	Comment
PARAMETERLIST	ANY_ARRAY_UINT	List of parameters: address and length. PARAMETERLIST is to be assigned to the corresponding array created by the MTM

NOTE: ParameterList is to be assigned to the ParamDesc variable taken from the configuration of the axis **recipe** in the *Movement* directory.

Description of the Output Parameters

The following table describes the output parameters that are in addition to the basic *(see page 33)* parameters:

Parameter	Type	Comment
PARAMETERSET	ANY_ARRAY_BYTE	Parameter buffer. PARAMETERSET is to be assigned to the corresponding array created by the MTM
SIZE	UINT	PARAMETERSET size in bytes.
ERRPARAMNUMBER	UDINT	Faulty index and subindex numbers.

NOTE: PARAMETERSET can be assigned to the Recipe variable if there is only one **recipe** declared for the axis.

TE_DOWNLOADDRIVEPARAM

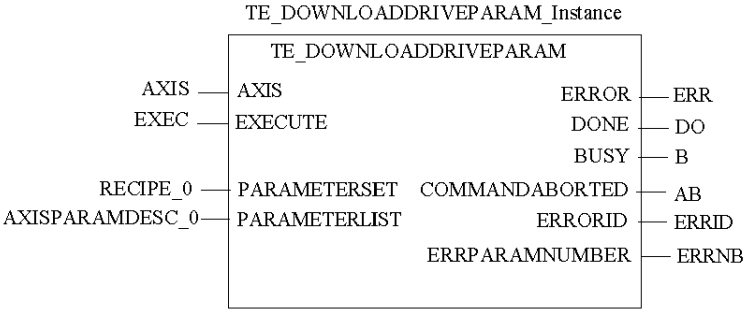
Function Description

The `TE_DOWNLOADDRIVEPARAM` function is used to transfer the parameters of a servodrive that have been previously saved in the PLC memory zone of the servodrive. The parameters have been saved for a change of production or for the replacement of a defective servodrive.

NOTE: Before executing this function, the servodrive status must be Disabled *(see page 29)*.

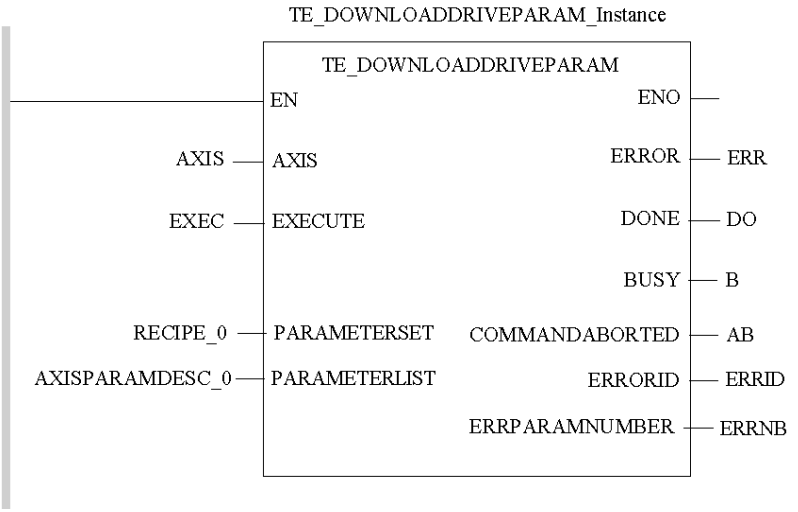
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
LD Axis
CAL TE_DOWNLOADDRIVEPARAM_Instance(AXIS:=AXIS, EXECUTE:=EXEC,  PARAMETE
RSET:=RECIPE_0, PARAMETERLIST:=AXISPARAMETERDESC_0, ERROR=>ERR, DONE=>D
O, BUSY=>B, COMMANDABORTED=>AB, ERRORID=>ERRID,  ERRPARAMNUMBER=>ERRNB)
```

Representation in ST

Representation:

```
TE_DOWNLOADDRIVEPARAM_Instance(AXIS:=AXIS, EXECUTE:=EXEC,  PARAMETERSET
:=RECIPE_0, PARAMETERLIST:=AXISPARAMETERDESC_0, ERROR=>ERR, DONE=>DO, B
USY=>B, COMMANDABORTED=>AB, ERRORID=>ERRID,  ERRPARAMNUMBER=>ERRNB);
```

Description of the Input Parameters

The following table describes the input parameters that are in addition to the basic *(see page 33)* parameters:

Parameter	Type	Comment
PARAMETERLIST	ANY_ARRAY_UINT	List of parameters: address and length. PARAMETERLIST is to be assigned to the corresponding array created by the MTM.
PARAMETERSET	ANY_ARRAY_BYTE	Parameter buffer. PARAMETERSET is to be assigned to the corresponding array created by the MTM

Description of the Output Parameters

The following table describes the output parameters that are in addition to the basic *(see page 33)* parameters:

Parameter	Type	Comment
ERRPARAMNUMBER	UDINT	Faulty index and subindex numbers.

LXM_UPLOADMTASK

Function Description

MotionTasks are created by a UniLink (by GMT) and saved directly to the EEPROM memory of the Lexium 15MP/LP/HP.

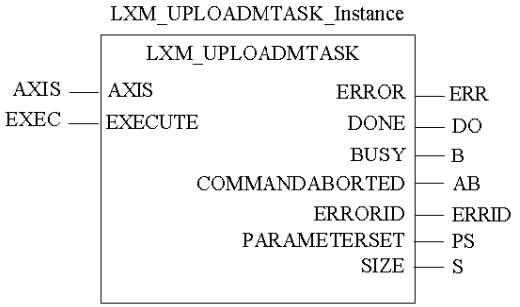
The `LXM_UPLOADMTASK` function is used to save the MotionTask parameters of the servodrive to a PLC memory zone.

This function is applicable on the Lexium 15MP/LP/HP servodrive only.

NOTE: Before executing this function, the servodrive status must be Disabled *(see page 29)*.

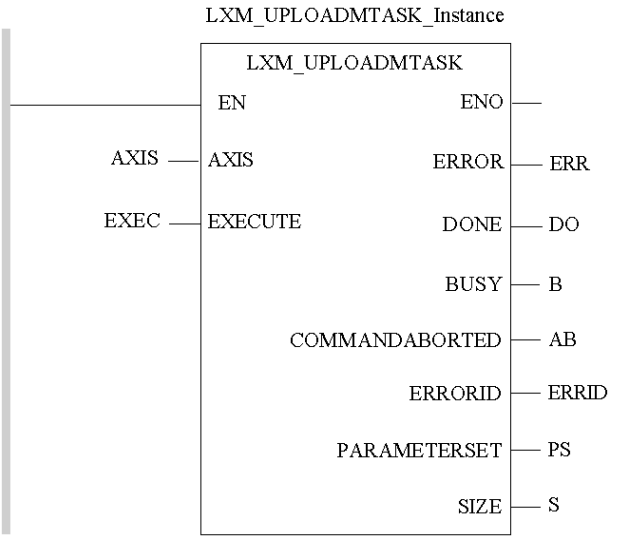
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
LD Axis
CAL LXM_UPLOADMTASK_Instance (AXIS:=AXIS, EXECUTE:=EXEC, ERROR=>ERR, DO
NE=>DO, BUSY=>B, COMMANDABORTED=>AB, ERRORID=>ERRID, PARAMETERSET=>PS,
SIZE=>S)
```

Representation in ST

Representation:

```
LXM_UPLOADMTASK_Instance (AXIS:=AXIS, EXECUTE:=EXEC, ERROR=>ERR, DONE=>
DO, BUSY=>B, COMMANDABORTED=>AB, ERRORID=>ERRID, PARAMETERSET=>PS, SIZE
=>S);
```

Description of the Output Parameters

The following table describes the output parameters that are in addition to the basic *(see page 33)* parameters:

Parameter	Type	Comment
PARAMETERSET	ANY_ARRAY_BYTE	The byte table where the Motion Tasks are saved (DDT structure). This table depends on the number of Motion Function Blocks used in the project. For Lexium 17/15MP the byte table is equal to: 12 + 27 MT number x bytes For Lexium 15LP the byte table is equal to: 12 + 29 MT number x bytes MT: Motion Task
SIZE	UINT	Size of the byte table corresponding to the PARAMETERSET parameter.

LXM_DOWNLOADMTASK

Function Description

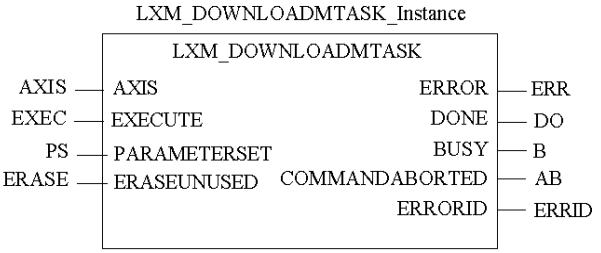
The `LXM_DOWNLOADMTASK` function is used to load a motion task from the memory of the PLC to the servodrive.

This function is applicable on the Lexium 15 servodrive only.

NOTE: before executing this function, the servodrive status must be Disabled *(see page 29)*.

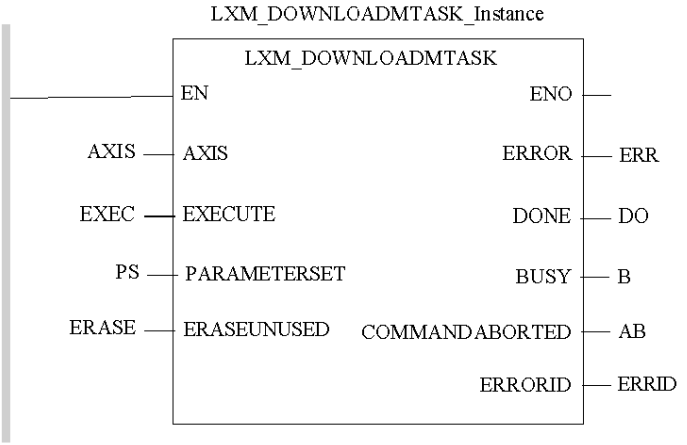
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
LD Axis
CAL LXM_DOWNLOADMTASK_Instance (AXIS:=AXIS, EXECUTE:=EXEC, PARAMETERSET
:=PS, ERASEUNUSED:=ERASE, ERROR=>ERR, DONE=>DO, BUSY=>B, COMMANDABORTED
=>AB, ERRORID=>ERRID)
```

Representation in ST

Representation:

```
LXM_DOWNLOADMTASK_Instance (AXIS:=AXIS, EXECUTE:=EXEC, PARAMETERSET:=PS
, ERASEUNUSED:=ERASE, ERROR=>ERR, DONE=>DO, BUSY=>B, COMMANDABORTED=>AB
, ERRORID=>ERRID));
```

Description of the Input Parameters

The following table describes the input parameter that is in addition to the basic *(see page 33)* parameters:

Parameter	Type	Comment
PARAMETERSET	ANY_ARRAY_BYTE	Motion tasks value (DDT structure).
ERASEUNUSED	BOOL	Erases unused tasks.

LXM_STARTMTASK

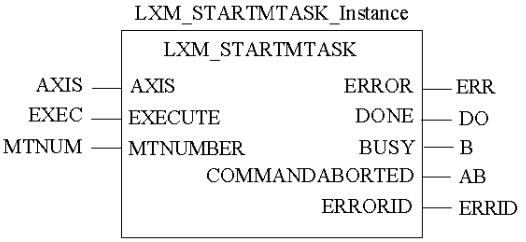
Function Description

The LXM_STARTMTASK function is used to launch the execution of the Motion Task for Lexium 15 whose number matches the value entered in the MTNUMBER input parameter.

The LXM_STARTMTASK function can also be used to launch the execution of the MotionSequence for Lexium 32.

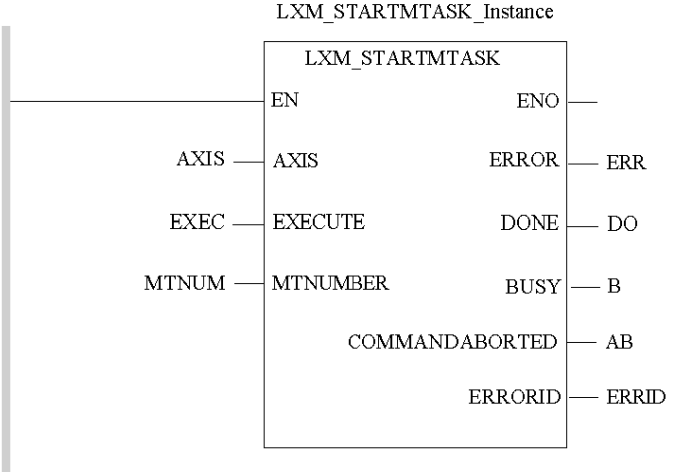
Representation in FBD

Representation:



Representation in LD

Representation:



Representation in IL

Representation:

```
LD Axis
CAL LXM_STARTMTASK_Instance(AXIS:=AXIS, EXECUTE:=EXEC, MTNUMBER:=MTNUM,
 ERROR=>ERR, DONE=>DO, COMMANDABORTED=>AB, BUSY=>B, ERRORID=>ERRID)
```

Representation in ST

Representation:

```
LXM_STARTMTASK_Instance(AXIS:=AXIS, EXECUTE:=EXEC, MTNUMBER:=MTNUM, ERR
OR=>ERR, DONE=>DO, COMMANDABORTED=>AB, BUSY=>B, ERRORID=>ERRID);
```

Description of the Input Parameters

The following table describes the input parameter that is in addition to the basic *(see page 33)* parameters:

Parameter	Type	Comment
MTNumber	UINT	Motion Task or MotionSequence number.

Units and Servodrives

Units

The following table shows you the speed, position and direction units by servodrive:

Units	Lexium15MP/HP/LP	Icla	ATV31	ATV71	Lexium05 / Lexium32
Position	PUNIT	inc	-	-	USR
Speed	VUNIT	rpm	rpm	rpm	rpm
Rotation	rpm	rpm	rpm	rpm	rpm
Torque	1/1000 rated torque	-	0.01 nominal torque	0.01 nominal torque	1/100 amp

USR is a unit that the user defines in Powersuite (by default, it is 16,384 units per rev).

PUNIT is a position unit that the user defines in Unilink.

VUNIT is a unit to be defined by Unilink (by default, this is PUNIT/sec).

NOTE: Rotation units only concern the `MC_MOVEVELOCITY` block.

Speed Equation of the Lexium 15 MP/HP in Speed Mode

The rotation speed of the motor controlled in speed mode by a Lexium 15MP is the setpoint speed multiplied by 0.5722 ($0.5722=60*10000/2^{20}$).

Example:

The value of the `VELOCITY` input parameter of the `MC_MOVEVELOCITY` block is 3000. The gearing rotation speed is therefore 1716.6 RPM (3000*0.5722).

Speed Equation of the Lexium 15 MP/HP in Position Mode

The rotation speed of an axis controlled in position mode by a Lexium 15MP is the setpoint speed multiplied by a coefficient K.

K=PGearI/10000.

PGearI is the numerator of the resolution configured in the servodrive by Unilink.

Example:

The value of the `VELOCITY` input parameter of the `MC_MOVEABSOLUTE` block is 3000. PGearI is 5000. Punit is in µm format.

K=5000/10000=0.5

Thus the axis speed is 15000 µm/sec (30000*0.5).

Speed Equation of the Lexium 15 LP in Position Mode and Speed Mode

Parameter VELOCITY equation of the Lexium 15 LP in Position mode for MC_MOVEABSOLUTE and MC_MOVERELATIVE :

VELOCITY (rpm) = VELOCITY (inc/250 µs) * (60*4000) / 2 ^ 32

Parameter VELOCITY equation of the Lexium 15 LP in a Velocity mode for MC_MOVEVELOCITY:

VELOCITY (rpm) = VELOCITY * 60 / P_GEARI

Unit of the status velocity of the Lexium 15 LP for MC_READACTUALVELOCITY:

PUNIT (Basic screen in Unilink) defines VELOCITY unit

Unit of the Position parameter and status of the Lexium 15 LP

VUNIT (Basic screen in Unilink) defines Position unit

We strongly advise the use of the speed units in rpm whatever the type of control is. In the other cases of units (i.e. mm/min) the formulas are to be adapted to take into account the linear dimension of the unit and no more the angular one, by integrating the P_GEARI factor there.

Tab synthesys => VELOCITY unit in rpm position mode using MC_MOVEABSOLUTE command for example:

VELOCITY Setpoint in rpm. Reading on Unilink in rpm

VELOCITY value returned by MC_READACTUALVELOCITY:

VELOCITY (inc/250 µs) = VELOCITY Setpoint (rpm) * 2 ^ 32 / (60 x 4000)

In velocity mode, using MC_MOVEVELOCITYcommand

VELOCITY Setpoint to be applied related to the velocity in rpm to be reached. Reading on Unilink in rpm

VELOCITY Setpoint = VELOCITY (rpm) * P_GEARI / 60

VELOCITY value returned by MC_READACTUALVELOCITY:

VELOCITY = VELOCITY Setpoint * 2 ^ 32 / (P_GEARI x 4000)

Messaging Service

Introduction

The blocks that use messaging slow down the PLC cycle time because they make `READ_VAR` and `WRITE_VAR` type requests to the servodrive.

Motion Function Block

The below table shows you which blocks use the messaging service, by type of servodrive:

Block name	Lexium 15MP/HP	Icla	ATV31	ATV71	Lexium05 / Lexium32	Lexium15 LP
MC_JOG	W 2024:7	W 3029:4 W 3029:5 W 3029:7 W 3029:8	NA	NA	W 3029:4 W 3029:5 W 3029:7 W 3029:8	NA
MC_READACTUALPOSITION	–	R position	–	–	–	–
MC_READACTUALTORQUE	–	NA	R 2002:6	R 6077	R 301E:3	R 6077
MC_READACTUALVELOCITY	–	R velocity	–	–	–	–
MC_STOP	R Stopstate	–	R Stopstate	R Stopstate	–	R Stopstate
MC_TORQUECONTROL	W 2060	NA	NA	W 6071 R 2002:39	no SDO	W 6071
MC_MOVEABSOLUTE MC_MOVERELATIVE MC_MOVEADDITIVE	W 6083:0 (ACC) W 6084:0 (DEC)	W 301D:1A (ACC)	–	–	W 6083:0 (ACC) W 6084:0 (DEC)	W 6083:0 (ACC) W 6084:0 (DEC)
MC_READAXISERROR	R 2070:16 R 1002:0 R 1003:1	R 301C:12 R 301C:A R 3020:7	R 2029:16 R 603F:0	R 2029:16 R 603F:0	R 301C:8 R 301C:C R 603F:0	R 385D:1 R 1002:0 R 1003:1
MC_HOME	W 607C:0 (Hposition) W 6099:1 (Hspeed) W 6098:0 (HType)	W 3028:B (Hposition) W 3028:4 (Hspeed)	–	–	W 3028:B (Hposition) W 6099:1 (Hspeed)	W 607C:0 (Hposition) W 6099:1 (Hspeed) W 6098:0 (HType)
LXM_GEARPOS LXM_GEARPOSS	W 3540:1 (GearI) W 353E:1 (GearO)	NA	NA	NA	no SDO	W 3540:1 W 353E:1
LXM_DOWNLOADMTASK	Equationf(MT)	–	–	–	–	Equationf(MT)
LXM_UPLOADMTASK	Equationf(MT)	–	–	–	–	Equationf(MT)
LXM_STARTMTASK	Task number	–	–	–	–	Task number

Block name	Lexium 15MP/HP	Icla	ATV31	ATV71	Lexium05 / Lexium32	Lexium15 LP
Notes	R: Use of READ_VAR messaging service W: Use of WRITE_VAR messaging service R 2029:16: Read an SDO-type object with index 2029 and subindex 16 W 3540:1: Write an SDO-type object with index 3540 and subindex 1 NA: Not applicable					

Memory Size

Memory Size of the Axis Declaration

The amount of memory used during one `AXIS_REF` axis declaration in the **Motion** directory without a recipe is 240 bytes.

Memory Size of the Recipe

The table below shows the memory size (in word) used by one or more recipes.

Device	Size used by the first recipe	Size used by the following recipe, with drive firmware current version	Size used by the following recipe, with drive firmware previous version
Lexium 15LP	1435	577	664
Lexium 15MP/HP	1268	510	
Lexium 32	1428	394	
ATV31	646	167	170
ATV71	1726	437	499
Lexium 05	314	96	133
Icla Ifa	303	99	
Icla Ife	244	82	
Icla Ifs	230	77	

Memory Size of MFB Blocks for Premium Platform

The table below shows on the Premium the memory size (in bytes) used by code, data by instance and by upload information in the MFB blocks.

MFB block type	Data by instance	Code
CAN_HANDLER	336	15229
LXM_DOWNLOADMTASK	112	3248
LXM_GEARPOSS	112	3360
LXM_GEARPOS	112	3296
LXM_STARTMTASK	96	2528
LXM_UPLOADMTASK	96	2288
MC_TORQUECONTROL	144	4336
MC_HOME	112	4686
MC_JOG	192	6784
MC_MOVEABSOLUTE	144	4752
MC_MOVEADDITIVE	144	3376

MFB block type	Data by instance	Code
MC_MOVERELATIVE	144	4096
MC_MOVEVELOCITY	128	3424
MC_POWER	80	1232
MC_READACTUALPOSITION	80	944
MC_READACTUALTORQUE	96	1110
MC_READACTUALVELOCITY	96	976
MC_READAXISERROR	96	2048
MC_READPARAMETER	96	1104
MC_READSTATUS	80	832
MC_RESET	80	2592
MC_STOP	128	3360
MC_WRITEPARAMETER	96	1248
TE_DOWNLOADDRIVEPARAM	176	5792
TE_UPLOADDRIVEPARAM	160	2352

NOTE: On Premium Legacy PLC (TSX571x4, TSX572x4, TSX573x4), the maximum code size for all EFB of the same family is limited to 64 K byte. You must check the code size used by MFB in your application to ensure that it does not involve a code size greater than 64 K byte. Otherwise it is not possible to generate the application.

Memory Size of MFB Blocks for Modicon M340 Platform

The table below shows on the Modicon M340 the memory size (in bytes) used by code, data by instance and by upload information in the MFB blocks.

MFB block type	Data by instance	Code
Axis_ref	248	
CAN_HANDLER	352	18992
LXM_DOWNLOADMTASK	128	3032
LXM_GEARPOS	128	3632
LXM_GEARPOSS	128	3840
LXM_STARTMTASK	110	2480
LXM_UPLOADMTASK	86	2832
MC_TORQUECONTROL	148	4816
MC_HOME	136	4688
MC_JOG	160	7984
MC_MOVEABSOLUTE	154	5056

MFB block type	Data by instance	Code
MC_MOVEADDITIVE	146	3552
MC_MOVERELATIVE	154	4320
MC_MOVEVELOCITY	144	3888
MC_POWER	78	1408
MC_READACTUALPOSITION	100	880
MC_READACTUALTORQUE	98	1264
MC_READACTUALVELOCITY	100	912
MC_READAXISERROR	116	2160
MC_READPARAMETER	102	1216
MC_READSTATUS	89	912
MC_RESET	80	2624
MC_STOP	143	3296
MC_WRITEPARAMETER	104	1216
TE_DOWNLOADDRIVEPARAM	176	7184
TE_UPLOADDRIVEPARAM	176	2240

Appendices

Overview

This section contains the appendicies.

What Is in This Appendix?

The appendix contains the following chapters:

Chapter	Chapter Name	Page
A	Error Codes and Values	113
B	MFB performances	117

Appendix A
Error Codes and Values

Tables of Error Codes for the Motion Function Block Library

Introduction

The tables presented in this section list the error codes and values generated by the blocks in the MotionFunctionBlock library.

Motion Function Blocks

The following table contains the error codes and values generated in the `ErrorId` output parameter of MFB blocks.

Error_Id (decimal format)	Meaning
0	Correct execution.
1	Premium and Atrium PLC messaging *(see Unity Pro, Communication, Block Library)* or M340 PLC messaging *(see Modicon M340 with Unity Pro, CANopen, User Manual)* system deferred error, to see details use MC_ReadAxisError *(see page 74)* block with the `MSGERRORID` information.
2	-
3	The block was interrupted due to a power cut.
4	Cancelled block (`Enable` switched to 0 during execution).
5	-
6	Faulty axis or axis missing.
7	`Axisref` changed value during execution of block or the same instance programmed with another `Axis_Ref`. If the value belongs to the `Can_Handler` block, the `Axis_Ref` is not intended for the `CAN_Handler` variable.
8	The servodrive has rejected the command (to see details, use `MC_ReadAxisError` block).
9	This block cannot be executed in this axis mode (axis mode available using `MC_ReadStatus`).
10	Bad condition for jog direction: Both forward and backward inputs are active. If the axis was moving, it will be stopped.
11	The block was interrupted by a `MC_Stop`.
12	The block was interrupted by another block (for example: one `MC_MOVE` interrupting another).
13	The block has stopped because the servodrive has taken too long to perform the command (e.g.: `MC_POWER`).

Error_Id (decimal format)	Meaning
14	The servodrive should be enabled but it is not now.
15	-
16	-
17	During an implicit exchange with the servodrive, the block has either: ● lost the semaphore because a priority block took it, ● or the block has not been invoked for n cycles, the system has therefore withdrawn the semaphore.
18	Unable to execute the MFB.
19	During an implicit exchange with the servodrive, the semaphore could not be obtained in the time allowed (too many blocks waiting for the servodrive, or servodrive too slow to react).
20	Internal error.
21	Parameter transfer error (corrupted data); `TE_UpLoadDriveParam` and `TE_DownLoadDriveParam` blocks.
22	During an implicit messaging exchange with the servodrive, the block has either: ● lost the semaphore because a priority block took it, ● or the block has not been invoked for n cycles, the system has therefore withdrawn the semaphore.
23	Internal error.
24	During a messaging exchange with the servodrive, the PLC's messaging system was saturated for more than five PLC cycles.
25	Internal error.
26	The AMT has configured a servodrive that the MFB cannot program.
27	PLC messaging *(see Unity Pro, Communication, Block Library)* system immediate error, see details using MC_ReadAxisError *(see page 74)* block with the MSGERRORID information.
28	Axis programmed in two `CAN_HANDLER`s.
29	InitAxis: No network at the programmed network address.
30	-
31	Internal error.
32	Temporary error sent back by `MC_Readstatus` during a warm restart.
33	`SET` variable is too small for the data to write.
34	`SET` variable is incompatible with this servodrive.
35	`LIST` variable is incompatible with this servodrive.
36	`LIST` variable corrupted (checksum error).
37	SET variable corrupted (checksum error).
38	Restarting of `Can_handler`

Error_Id (decimal format)	Meaning
39	The **TSX CPP 110** card does not see the device at the configured address.
40	The family in the configuration does not match the one in the current servodrive.
41	The reference in the configuration does not match the one in the current servodrive.
42	Servodrive unknown.
43	The parameters `List` version does not match the `Set` version set of parameters.
44	The servodrive is still faulty after a reset.
45	The device does not work correctly when the MFB command is performed (only for Lexium 17).

Appendix B
MFB performances

MFB Performance Table

At a glance

The table below presents for the differents tests with Motion Function Block the performances for each servodrive.

Conditions and rules

CANopen baud rate configuration = 500kbds

PLC period task is :

- 5 ms+ nb_axis_x1ms if nb_axis>3,
- 5 ms if nb_axis<=3.

Test table

The test are done for the following servodrives: Lexium 05 / Lexium 32, Lexium 15 MP/HP, Lexium 15 LP, Icla, ATV31 and ATV71.

	Performances (ms)					
Test Type	**Lexium 05 / Lexium 32**	**Lexium 15 MP/HP**	**Lexium 15 LP**	**Icla**	**ATV31**	**ATV71**
`MC_MOVEABSOLUTE` Start (change mode, modification of acceleration and deceleration)	(6 to 7)c	(6 to 7)c	(6 to 7)c	(4 to 5)c	X	X
`MC_MOVEABSOLUTE` Start (no changes)	1c	1c	1c	1c	X	X
Time between 2 `MC_MOVEABSOLUTE` (1)	2c	2c + 3 ms + TcMH	2c + 1 ms + TcLP	2c	X	X
Time between 2 `MC_MOVEABSOLUTE` (second motion task just after the first motion task)	1c + 0.5 ms	1c + 3 ms + TcMH	1c + 1 ms + TcLP	1c + 0.5 ms	X	X
Interruption delay `MC_MOVEABSOLUTE` to the same instance	1c + 0.5 ms	1c + 5 ms + TcMH	1c + 1 ms + TcLP	1c + 0.5 ms	X	X

	Performances (ms)					
Test Type	**Lexium 05 / Lexium 32**	**Lexium 15 MP/HP**	**Lexium 15 LP**	**Icla**	**ATV31**	**ATV71**
Interruption delay `MC_MOVEABSOLUTE` with another instance	1c + 0.5 ms	1c + 5 ms + TcMH	1c + 1 ms + TcLP	1c + 0.5 ms	X	X
Start motion task	X	1c + 5 ms + TcMH	1c + 2 ms + TcLP	X	X	X
Delay between two motion tasks (1)	X	1c + 5 ms + TcMH	42 + TcLP	X	X	X
Stop during Move	1c + 0.5 ms	1c + 1 ms + TcMH	1c + 2 ms + TcLP	1c + 0.5 ms	X	X
Change speed in the same instance of `MC_MOVEVELOCITY`	1c + 0.5 ms	1c + 1 ms + TcMH	1c + 1 ms + TcLP	1c + 0.5 ms	1c + 10 ms + TcATV31	1c + 5 ms + TcATV71
Save servodrives parameters (`TE_UPLOADDRIVEPARAM`)	212	760	851	197	473	1283
Transfer of servodrives parameters (`TE_DOWNLOADDRIVEPARAM`)	219	1239	1460	204	502	1329
Save 20 motion tasks (`LXM_UPLOADMTASK`)	X	1778	1739	X	X	X
Load 20 motion tasks (`LXM_DOWNLOADMTASK`)	X	1237	583	X	X	X
Legend						
(1): in the case of a program where the second comand comes before the first command, you have to add one PLC period task on the equation. c: PLC period task TcMH: Lexium 15 MP/HP cycle time (0 to 3 ms) TcLP: Lexium 15 LP cycle time (0 to 1 ms) TcATV31: ATV31 cycle time (15 ms) TcATV71: ATV71 cycle time (5 ms)						

Examples

Applications with 3 axis (Lexium 05):

c = 5 ms

Time start a `MC_MOVEABSOLUTE` with no changes = 1c = 5 ms

Interruption delay of the same instance of `MC_MOVEABSOLUTE` = 1c+0,5ms = 5+0,5 = 5,5 ms

Applications with 1 axis (Lexium 15LP):

c = 5 ms

Time start a `MC_MOVEABSOLUTE` with changes = 6c +1c = 6x5+5 = 35 ms

Glossary

B

BOOL

`BOOL` is the abbreviation for the Boolean type. This is the basic data type in computing. A `BOOL` variable can have either of the following two values: 0 (`FALSE`) or 1 (`TRUE`).

A bit extracted from a word is of type `BOOL`, for example: `%MW10.4`.

D

DINT

`DINT` is the abbreviation of Double INTeger (encoded in 32 bits).

The upper/lower limits are as follows: -(2 to the power of 31) to (2 to the power of 31) - 1.

Example:

`-2147483648`, `2147483647`, `16#FFFFFFFF`.

I

INT

`INT` is the abbreviation of single INTeger (encoded in 16 bits).

The upper/lower limits are as follows: -(2 to the power of 15) to (2 to the power of 15) - 1.

Example:

`-32768`, `32767`, `2#1111110001001001`, `16#9FA4`.

IODDT

IODDT is the abbreviation of Input/Output Derived Data Type.

The term IODDT indicates a structured data type representing a module or a channel of a PLC module. Each expert module has its own IODDTs.

U

UDINT

UDINT is the abbreviation of Unsigned Double INTeger (encoded in 32 bits). The upper/lower limits are as follows: 0 to (2 to the power of 32) - 1.

Example:

0, 4294967295, 2#11111111111111111111111111111111, 8#37777777777, 16#FFFFFFFF.

UINT

UINT is the abbreviation of the Unsigned INTeger format (encoded in 16 bits). The upper/lower limits are as follows: 0 to (2 to the power of 16) - 1.

Example:

0, 65535, 2#1111111111111111, 8#177777, 16#FFFF.

Index

A

availability of the instructions, *23*
AXIS_REF, *28*

B

basic parameters, *33*

C

CAN_HANDLER, *38*

I

instructions
availability, *23*

L

Lexium-instructions
LXM_DOWNLOADMTASK, *99*
LXM_GEARPOS, *84*
LXM_GearPosS, *87*
LXM_STARTMTASK, *101*
LXM_UPLOADMTASK, *96*
LXM_DOWNLOADMTASK, *99*
LXM_GEARPOS, *84*
LXM_GearPosS, *87*
LXM_STARTMTASK, *101*
LXM_UPLOADMTASK, *96*

M

MC_HOME, *82*
MC_JOG, *70*
MC_MOVEABSOLUTE, *60*
MC_MOVEADDITIVE, *64*
MC_MOVERELATIVE, *62*
MC_MOVEVELOCITY, *68*
MC_POWER, *58*
MC_READACTUALPOSITION, *45*
MC_READACTUALTORQUE, *49*
MC_READACTUALVELOCITY, *47*
MC_READAXISERROR, *74*
MC_READPARAMETER, *40*
MC_READSTATUS, *78*
MC_RESET, *53*
MC_STOP, *55*
MC_TORQUECONTROL, *51*
MC_WRITEPARAMETER, *43*
motion-instructions
CAN_HANDLER, *38*
MC_HOME, *82*
MC_JOG, *70*
MC_MOVEABSOLUTE, *60*
MC_MOVEADDITIVE, *64*
MC_MOVERELATIVE, *62*
MC_MOVEVELOCITY, *68*
MC_POWER, *58*
MC_READACTUALPOSITION, *45*
MC_READACTUALTORQUE, *49*
MC_READACTUALVELOCITY, *47*
MC_READAXISERROR, *74*
MC_READPARAMETER, *40*
MC_READSTATUS, *78*
MC_RESET, *53*
MC_STOP, *55*
MC_TORQUECONTROL, *51*
MC_WRITEPARAMETER, *43*
TE_DOWNLOADDRIVEPARAM, *93*
TE_UPLOADDRIVEPARAM, *90*

S

status chart, *29*

T

TE_DOWNLOADDRIVEPARAM, *93*
TE_UPLOADDRIVEPARAM, *90*